# ReWA.AI

Complete source code

github.com/jeevandj2008s-byte/rewa-ai

16 October 2026

**Files included:**


.gitignore  (11 lines)
.streamlit/config.toml  (2 lines)
README.md  (189 lines)
app1.py  (3971 lines)
auth.py  (412 lines)
data_base.py  (125 lines)
digital_twin.py  (181 lines)
recycling_logic.py  (50 lines)
requirements.txt  (10 lines)
rewa_logic.py  (227 lines)
rewabuddy_advisor.py  (785 lines)
rewards.py  (313 lines)
test_rewa.py  (234 lines)

## .gitignore

```
.streamlit/secrets.toml
**/secrets.toml
**/secreats.toml
__pycache__/
*.pyc
*.db
.venv/
venv/
.env*
.vercel
.vscode/
```

## .streamlit/config.toml

```
[server]
enableStaticServing = true
```

## README.md

```
# ?? ReWA.AI - Recycling-Reusable Waste Assistant

**Sort Today * Save Tomorrow**

 ReWA.AI helps you answer this simple question: **"What is the waste? How can I recycle/reuse it? which bin should the waste go?"**

 You take a photo of a waste item. The app uses AI to guess what it is, suggests the right bin, and guides you how to recycle it, dispose of it and reuse it. Every item you sort earns virtual **Eco Points**, so recycling feels like a game.

> ?
    > [ReWA.AI Homepage](<img width="1887" height="912" alt="Screenshot 2026-10-16 152509" src="https://github.com/user-attachments/assets/eaa25895-1696-4426-98bd-b4d37f89bf75" />
)
    > [AI-Powered Waste Assistant](<img width="1897" height="906" alt="Screenshot 2026-10-16 152556" src="https://github.com/user-attachments/assets/cb9c5e97-1bf3-4229-8caf-e5fc145e3697" />
)
    > [Smart Recycling Guidance](<img width="1896" height="901" alt="Screenshot 2026-10-16 152614" src="https://github.com/user-attachments/assets/d3a63aa9-25f2-4e76-bbed-7e76e0a620fd" />
)

> [Digital Twin - Virtual Waste Bins](<img width="1912" height="896" alt="Screenshot 2026-10-16 152745" src="https://github.com/user-attachments/assets/b509d3e1-fd8f-4c74-bd86-e81863dfe97a" />
)
> [Analytics and Recycling Statistics](<img width="1897" height="891" alt="Screenshot 2026-10-16 152710" src="https://github.com/user-attachments/assets/cebbd70d-776d-47f9-bbb1-fb92a45debc1" />
)
    > [Eco Points and Daily Streak](<img width="1910" height="900" alt="Screenshot 2026-10-16 152824" src="https://github.com/user-attachments/assets/ca0af40a-1098-4ab1-96dd-9af344b002d3" />
)
)

---

## ? What can it do?
| ? **Photo classification** | Upload photos or use your live camera. AI guesses the item (bottle, can, battery, food waste...). |
| ? **ReWA AI Buddy** | An AI helper that explains the recycling advice and gives three easy reuse ideas in friendly language. |
| ?? **Six virtual bins** | Plastic, Paper, Metal, Glass, Hazardous and Biodegradable. You always confirm the bin before adding an item. |
| ?? **Hazardous warnings** | Batteries, medicines, needles, chemicals and similar items are flagged and always go to the Hazardous bin, with extra-safe advice. |
| ? **Eco Points** | 10 virtual points for every item you record. |
| ? **Streaks and daily goal** | Recycle on consecutive days to build a streak. Daily goal: 5 items. |
| ? **Badges, garden and passport** | 7 badges, a garden that grows as you recycle, and a passport with a stamp for each bin. |
| ? **Analytics** | Charts of items per material, bin capacity, points by category and the last 7 days. |
| ? **Sign-in (optional)** | Log in with Google or a phone number so each person keeps their own records. |

---

## ? How does it work?

1. **You add a photo.** Upload files (JPG, PNG, WEBP) or take a picture with your camera. Up to 10 images at a time.
2. **The AI looks at it.** The app uses a model called **CLIP** (`openai/clip-vit-large-patch14`). It compares your photo with a list of about 22 waste labels and picks the best matches. The score it shows is a similarity score, **not** a guarantee that it is right.
3. **You check the bin.** The app suggests a bin. If the AI is not sure (score below 55%), it asks you to check manually.
4. **You add it to the bin.** The item is saved in the database and you earn Eco Points.
5. **Ask ReWA Buddy (optional).** The app sends the *checked facts* to an AI writer (Gemini, then Groq, then OpenRouter as backups). If all of them fail, it uses built-in Python rules instead, so you always get advice.

**Important design choice:** Python decides the material type, the bin and the safety rules. The AI writer only explains them in simple words. This stops the AI from inventing facts, especially for dangerous items.

---
```

```
## ?? Honest notes

- **AI can be wrong.** Always check the real material and your local recycling rules.
- **Bin levels are simulated.** Each item adds 5% to a bin. There are no sensors or hardware.
- **Eco Points are virtual.** They have no cash value, and recording an item does not prove it was really recycled.

---

## ?? Project files

| File | What it does |
|---|---|
| `app.py` | The main Streamlit app: pages, design, uploads, charts. Run this file. |
| `auth.py` | Sign-in with Google or phone number (optional). |
| `data_base.py` | Saves and reads recycling events in SQLite (`recycling.db`). |
| `rewards.py` | Eco Points, daily activity and streaks. Makes sure the same item is never rewarded twice. |
| `digital_twin.py` | The six virtual bins: counting items, simulated capacity, emptying. |
| `recycling_logic.py` | Decides which bin an item goes in and whether that bin is nearly full. |
| `rewa_logic.py` | Helper logic with no Streamlit: image checks, badges, garden, missions, passport. |
| `rewabuddy_advisor.py` | The ReWA AI Buddy: reuse ideas, safety tips and the Gemini ? Groq ? OpenRouter fallback. |
| `test_rewa_logic.py` | Automatic tests for `rewa_logic.py`. |

---

## ? How to run it

### 1. Get the code
```bash
git clone https://github.com/<your-username>/<your-repo>.git
cd <your-repo>
```

### 2. Install the requirements
You need **Python 3.10+**.
```bash
pip install -r requirements.txt
```

### 3. (Optional) Add your AI keys
Without keys the app still works. ReWA Buddy just uses built-in rules.
```bash
export GEMINI_API_KEY="your-key"
export GROQ_API_KEY="your-key"
export OPENROUTER_API_KEY="your-key"
```
On Windows PowerShell use `$env:GEMINI_API_KEY="your-key"`.

### 4. Start the app
```bash
streamlit run app1.py
```
The first run downloads the CLIP model, so it can take a minute and needs internet.

---

## ? Deploy it publicly (Streamlit Community Cloud)

ReWA.AI is a Streamlit app, so it is hosted on Streamlit Community Cloud (free), not Vercel.

1. Go to https://share.streamlit.io and sign in with GitHub.
2. Click **Create app** ? **Deploy a public app from GitHub**.
3. Fill in:
   - **Repository:** `jeevandj2008s-byte/rewa-ai`
   - **Branch:** `main`
   - **Main file path:** `app1.py`
   - **App URL:** pick a name, e.g. `rewa-ai`
4. Open **Advanced settings**:
   - **Python version:** 3.11
   - **Secrets:** paste the contents of your local `.streamlit/secrets.toml`, changing
     `redirect_uri` to `https://<your-app-name>.streamlit.app/oauth2callback`.
     Add AI keys here too if you use them, e.g. `GEMINI_API_KEY = "your-key"`.
5. Click **Deploy**. The first build installs torch and downloads CLIP, so it takes a few minutes.
   The app uses the smaller `openai/clip-vit-base-patch32` model by default, because the large
```

```
   model runs out of memory on Streamlit Cloud. To use a different model, add
   `REWA_MODEL = "openai/clip-vit-large-patch14"` to Secrets (only on a server with enough RAM).
6. In Google Cloud Console, add the same `https://<your-app-name>.streamlit.app/oauth2callback`
   to your OAuth client's **Authorized redirect URIs**, or Google sign-in will fail.

 The app is public by default - anyone with the link can open it. Every `git push` to `main` redeploys it automatically.

> Note: the SQLite database lives on the server's disk, which Streamlit Cloud resets on restart/redeploy, so saved points and history can be lost. Use a hosted database if you need them to persist.

---

## ? Setting up sign-in

Create `.streamlit/secrets.toml`:
```toml
[auth]
redirect_uri = "http://localhost:8501/oauth2callback"
cookie_secret = "any-long-random-string"
client_id = "xxxx.apps.googleusercontent.com"
client_secret = "xxxx"
server_metadata_url = "https://accounts.google.com/.well-known/openid-configuration"

# Optional phone login (SMS code through Twilio Verify)
[twilio]
account_sid = "ACxxxx"
auth_token = "xxxx"
verify_sid = "VAxxxx"
```
- Google: create an OAuth Web client in Google Cloud Console and add the same `redirect_uri` there.
- Each user's items, points and streaks are stored separately by `user_id`.
- **Never upload `secrets.toml` to GitHub.**

---

## ? Run the tests
```bash
python -m unittest -v test_rewa_logic
```
The tests check reward logic (no double points), image validation, badges, garden stages and missions.

---

## ?? Built with

- **Python** and **Streamlit** (web app)
- **Hugging Face Transformers** with **CLIP** (image understanding)
- **Pillow** (image handling), **pandas** and **Altair** (charts)
- **SQLite** (database)
- **Google Gemini**, **Groq** and **OpenRouter** (advice writing)

---

## ? Ideas for the future
- Real bin sensors instead of simulated capacity
- Local recycling rules by city
- Team or school leaderboards
- A cloud database (for example Postgres) for many users

---

## ??? Author
 Made by **Jeevan DJ and J.Jebastin Immanuvel** - *Bangalore Institution of Technology-Information Science Engineering(1st Year)*

## ? License
This project is licensed under the MIT License.
```

## app1.py

```
import base64
import hashlib
import io
import logging
import os
import random
import sqlite3
import uuid
from pathlib import Path

import altair as alt
import pandas as pd
import streamlit as st
from PIL import Image
from transformers import pipeline

import auth
import data_base
import rewa_logic
import rewards as rewards_module
from data_base import create_data_base, get_events, get_bin_totals
from digital_twin import create_bins, record_item, empty_bin, get_bin_for_item
from rewabuddy_advisor import generate_recycling_advice
from rewards import (
    DAILY_GOAL,
    POINTS_PER_EVENT,
    award_event,
    get_daily_history,
    get_points_by_category,
    get_summary,
    init_rewards_db,
)

# ---------- PAGE SETTINGS ----------
st.set_page_config(
    page_title="ReWA.AI",
    page_icon="??",
    layout="wide"
)

# Everyone can open the site. A visitor who has not logged in is a "guest":
# they can try the classifier and fill the virtual bins for this visit, but
# nothing is saved and no Eco Points are earned. Logging in (the Log in panel at the top)
# lets them save items and earn Eco Points.
# (set_page_config must stay the first Streamlit call, so this goes after it.)
current_user = auth.resolve_user()
data_base.set_current_user(current_user["id"])
is_guest = current_user["is_guest"]

create_data_base()
init_rewards_db()

# ---------- GREENERY PALETTE ----------
# Forest, emerald, sage and cream. (The CSS below defines the same colours as
# CSS variables; these Python constants are used by charts and bin cards.)
DEEP = "#06241A"        # deepest forest: intro, hero, footer
EMERALD = "#0C4A34"     # emerald: gradients
FOREST = "#14432F"      # primary: headings
LEAF = "#2E7D52"        # mid green
SAGE = "#9DB89B"        # soft sage
GOLD = "#C9A24B"        # accent: hairlines, highlights
CHAMPAGNE = "#F1E3BC"   # soft gold
MINT = "#A8E6C0"        # success / glow
IVORY = "#F7F3E7"       # cream page background
PALE_LEAF = "#E6EFE0"   # cards, subtle panels
INK = "#12241A"         # body text

# One consistent colour per bin, used by every chart and bin card
BIN_COLORS = {
    "Plastic": "#2F80ED",
    "Paper": "#E0A526",
```

```
    "Metal": "#7A8794",
    "Glass": "#1FA39A",
    "Hazardous": "#D64545",
    "Biodegradable": "#4CAF50",
}
BIN_ICONS = {
    "Plastic": "??",
    "Paper": "?",
    "Metal": "?",
    "Glass": "?",
    "Hazardous": "??",
    "Biodegradable": "?",
}

# ---------- IMAGE INPUT SETTINGS ----------
# The AI model runs on your computer, so a very large batch can be slow.
# Raise or lower this number to suit your hardware.
MAX_IMAGES = 10

# The opening landing animation (about 5 seconds, plays once per visit).
# True = play it. Set to False if you ever want to skip it.
PLAY_INTRO = True

# Technical error details are hidden from visitors (they can contain file
# paths or service URLs). Start the app with  REWA_DEBUG=1  to show them.
DEBUG = os.environ.get("REWA_DEBUG") == "1"
logger = logging.getLogger("rewa")


def show_error_details(error):
    """Log the real error; show it on screen only in debug mode."""
    logger.warning("Error details: %s", error)
    if DEBUG:
        st.caption(f"Technical details: {error}")

# ---------- WASTE LABELS ----------
# Labels the AI model chooses from. Hazardous items have several labels so
# they are no longer swallowed by "other waste".
CANDIDATE_LABELS = [
    "cardboard box",
    "paper sheet",
    "newspaper",
    "plastic bottle",
    "plastic bag",
    "plastic wrapper",
    "metal can",
    "glass bottle",
    "glass jar",
    "food waste",
    "fruit peel",
    # --- hazardous / medical ---
    "battery",
    "light bulb",
    "medicine",
    "syringe or needle",
    "medical waste",
    "biohazard waste",
    "chemical container",
    "aerosol can",
    "paint can",
    "pesticide or poison",
    # --- catch-all ---
    "other waste",
]

# Any label containing one of these words is treated as hazardous.
# (Labels are compared in lowercase with underscores instead of spaces.)
HAZARDOUS_KEYWORDS = [
    "battery",
    "light_bulb",
    "medicine",
    "syringe",
    "needle",
    "sharps",
    "medical",
```

```
    "biohazard",
    "chemical",
    "aerosol",
    "paint",
    "pesticide",
    "poison",
    "hazardous",
]

# Subset of the above that needs the special "sharps / medical" advice.
SHARPS_KEYWORDS = ["syringe", "needle", "sharps", "medical", "biohazard"]


def normalize_label(label):
    return label.lower().replace(" ", "_")


def is_hazardous_label(label):
    label = normalize_label(label)
    return any(word in label for word in HAZARDOUS_KEYWORDS)


def resolve_bin(label):
    """Hazardous items go to the Hazardous bin; otherwise use the original mapping."""
    if is_hazardous_label(label):
        return "Hazardous"
    return get_bin_for_item(label)


# ---------- BANNER IMAGE (OPTIONAL) ----------
# The hero is drawn with HTML/CSS/SVG, so it stays sharp on every screen.
# If you would rather show your own exported banner picture instead, save it
# next to this file as rewa_banner.jpg / .jpeg / .png and set this to True.
USE_BANNER_IMAGE = False
BANNER_FILE_NAMES = ["rewa_banner.jpg", "rewa_banner.jpeg", "rewa_banner.png"]


def load_banner_base64():
    """Return (mime_type, base64_string) for the banner image, or (None, None)."""
    try:
        base_dir = Path(__file__).resolve().parent
    except NameError:
        base_dir = Path.cwd()

    for name in BANNER_FILE_NAMES:
        path = base_dir / name
        if path.exists():
            mime = "image/png" if path.suffix.lower() == ".png" else "image/jpeg"
            return mime, base64.b64encode(path.read_bytes()).decode()
    return None, None


# ---------- GRAPHICS (inline SVG) ----------
# Recycling arrows + leaf mark used in the intro and the hero.
LOGO_MARK_SVG = (
    '<svg class="rewa-mark" viewBox="0 0 120 120" xmlns="http://www.w3.org/2000/svg">'
    '<defs>'
    '<linearGradient id="mkA" gradientUnits="userSpaceOnUse" x1="14" y1="20" x2="100" y2="108">'
    '<stop offset="0%" stop-color="#D8F8B6"/>'
    '<stop offset="100%" stop-color="#8FDB7A"/>'
    '</linearGradient>'
    '<linearGradient id="mkB" x1="0" y1="0" x2="1" y2="1">'
    '<stop offset="0%" stop-color="#EAFBC8"/>'
    '<stop offset="100%" stop-color="#9BE08A"/>'
    '</linearGradient>'
    '</defs>'
    '<g fill="url(#mkA)" stroke="url(#mkA)" stroke-linejoin="round" stroke-linecap="round">'
    '<path d="M54 22 L30 64" stroke-width="12" fill="none"/>'
    '<polygon points="19,83 42,71 18,57" stroke-width="3"/>'
    '<g transform="rotate(-120 60 66)">'
    '<path d="M54 22 L30 64" stroke-width="12" fill="none"/>'
    '<polygon points="19,83 42,71 18,57" stroke-width="3"/>'
    '</g>'
    '</g>'
    '<path d="M94 78 Q104 42.6 68 33 Q58.6 68.2 94 78 Z" fill="url(#mkB)"/>'
```

```
    '<path d="M90 73 L74 41" stroke="#1F6F3A" stroke-width="2.4" '
    'fill="none" stroke-linecap="round" opacity="0.8"/>'
    '</svg>'
)

# Same mark with its own gradient ids, so it can appear twice on one page.
INTRO_MARK_SVG = LOGO_MARK_SVG.replace("mkA", "iA").replace("mkB", "iB")

# Small leaf used in the divider line.
DIVIDER_LEAF_SVG = (
    '<svg class="rule-leaf" viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">'
    '<path d="M3 21 C3 9 11 3 22 3 C22 14 16 21 3 21 Z" fill="#E3F8D0"/>'
    '<path d="M3 21 L15 9" stroke="#1F6F49" stroke-width="1.4" fill="none" '
    'stroke-linecap="round"/>'
    '</svg>'
)

# A single leaf shape; its colour comes from CSS (fill: currentColor).
LEAF_SHAPE_SVG = (
    '<svg viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg" aria-hidden="true">'
    '<path d="M3 21 C3 9 11 3 22 3 C22 14 16 21 3 21 Z"/>'
    '<path d="M3 21 L15 9" stroke="rgba(255,255,255,.45)" stroke-width="1.1" '
    'fill="none" stroke-linecap="round"/>'
    '</svg>'
)

LEAF_COLORS = ["#2E7D52", "#4E9A68", "#9DB89B", "#14432F", "#7CB98A", "#C9A24B"]
INTRO_LEAF_COLORS = ["#A8E6C0", "#D8F8B6", "#8FDB7A", "#4E9A68", "#F1E3BC"]
HERO_LEAF_COLORS = ["#A8E6C0", "#D8F8B6", "#8FDB7A", "#6CC08B", "#F1E3BC"]


def leaf_field(count, seed, size=(18, 44), duration=(18, 34),
               delay=(-30, 0), colors=None, spread_y=False):
    """Return `count` leaf <span>s with randomised (but repeatable) positions.

    Each leaf gets CSS variables (--x, --y, --s, --d, --dl, --c); the CSS
    decides how it moves (drifting, rising or floating).
    """
    rnd = random.Random(seed)
    colors = colors or LEAF_COLORS
    parts = []
    for _ in range(count):
        style = f"--x:{rnd.randint(2, 96)}%;"
        if spread_y:
            style += f"--y:{rnd.randint(4, 86)}%;"
        style += (
            f"--s:{rnd.randint(*size)}px;"
            f"--d:{rnd.uniform(*duration):.1f}s;"
            f"--dl:{rnd.uniform(*delay):.1f}s;"
            f"--c:{rnd.choice(colors)}"
        )
        parts.append(f'<span class="leaf" style="{style}">{LEAF_SHAPE_SVG}</span>')
    return "".join(parts)


def fireflies(count, seed):
    """Tiny glowing dots that twinkle (decorative)."""
    rnd = random.Random(seed)
    return "".join(
        f'<span class="fly" style="--x:{rnd.randint(3, 97)}%;'
        f'--y:{rnd.randint(5, 95)}%;--d:{rnd.uniform(2.5, 5):.1f}s;'
        f'--dl:{rnd.uniform(0, 3):.1f}s"></span>'
        for _ in range(count)
    )


# Four outlined icon tiles on the right-hand side of the hero.
_ICON_ATTR = (
    'viewBox="0 0 24 24" fill="none" stroke="#EAF9DF" stroke-width="1.5" '
    'stroke-linecap="round" stroke-linejoin="round" xmlns="http://www.w3.org/2000/svg"'
)
TILES_HTML = (
    '<div class="tiles">'
    f'<div class="tile"><svg {_ICON_ATTR}>'
    '<path d="M10 2h4v3l1.5 2.5V21a1 1 0 0 1-1 1h-5a1 1 0 0 1-1-1V7.5L10 5z"/>'
```

```
    '<path d="M8.5 11h7M8.5 16h7"/></svg></div>'
    f'<div class="tile"><svg {_ICON_ATTR}>'
    '<path d="M6 5c0-1.1 2.7-2 6-2s6 .9 6 2v14c0 1.1-2.7 2-6 2s-6-.9-6-2z"/>'
    '<path d="M6 5c0 1.1 2.7 2 6 2s6-.9 6-2M6 9c0 1.1 2.7 2 6 2s6-.9 6-2"/></svg></div>'
    f'<div class="tile"><svg {_ICON_ATTR}>'
    '<path d="M7 19H4.815a1.83 1.83 0 0 1-1.57-.881 1.785 1.785 0 0 1-.004-1.784L7.196 9.5"/>'
    '<path d="M11 19h8.203a1.83 1.83 0 0 0 1.556-.89 1.784 1.784 0 0 0 0-1.775l-1.226-2.12"/>'
    '<path d="m14 16-3 3 3 3"/>'
    '<path d="M8.293 13.596 7.196 9.5 3.1 10.598"/>'
    '<path d="m9.344 5.811 1.093-1.892A1.83 1.83 0 0 1 11.985 3a1.784 1.784 0 0 1 1.546.888l3.943 6.843"/>'
    '<path d="m13.378 9.633 4.096 1.098 1.097-4.096"/></svg></div>'
    f'<div class="tile"><svg {_ICON_ATTR}>'
    '<path d="M4 20C4 10 10 4 20 4c0 10-6 16-16 16z"/><path d="M4 20L14 10"/></svg></div>'
    '</div>'
)

# ---------- SESSION STATE ----------
# List of analysed images. Each entry is a dict:
#   uid, name, image (PIL), results (top-3 predictions), best_label, best_score
if "batch" not in st.session_state:
    st.session_state.batch = []

# uids of items that were already recorded in a recycling bin
if "recorded_uids" not in st.session_state:
    st.session_state.recorded_uids = set()

# Which bin each recorded item went into: {uid: bin_name}
if "recorded_bins" not in st.session_state:
    st.session_state.recorded_bins = {}

# Eco Points that failed to save (e.g. database busy): {uid: (label, bin)}.
# They are retried automatically; the database refuses double rewards.
if "pending_rewards" not in st.session_state:
    st.session_state.pending_rewards = {}

# Fingerprints of photos already recorded this visit, so the same picture
# cannot be classified again and counted as a second item.
if "recorded_fingerprints" not in st.session_state:
    st.session_state.recorded_fingerprints = set()

# Photos taken with the live camera, waiting to be classified:
#   [{"id": md5, "name": "Camera photo 1", "data": bytes}, ...]
if "camera_shots" not in st.session_state:
    st.session_state.camera_shots = []

# Changing these numbers gives the uploader / camera a fresh, empty widget
if "uploader_key" not in st.session_state:
    st.session_state.uploader_key = 0

if "camera_key" not in st.session_state:
    st.session_state.camera_key = 0

def rebuild_bins_from_database():
    """Create fresh virtual bins and fill them from the saved event log.

    Bin capacity is SIMULATED (item count x a fixed step, capped at 100%).
    No sensor or hardware is connected.
    """
    fresh = create_bins()
    try:
        saved_totals = get_bin_totals()
    except Exception as error:
        logger.warning("Could not read saved bin totals: %s", error)
        saved_totals = {}

    for material, total in saved_totals.items():
        if material in fresh:
            fresh[material]["item_count"] = total
            fresh[material]["capacity_percent"] = min(total * 5, 100)

    st.session_state.digital_twin_bins = fresh


if "digital_twin_bins" not in st.session_state:
    rebuild_bins_from_database()
```

```

if "flash" not in st.session_state:
    st.session_state.flash = None

# True while the "Are you sure?" step of the Eco Points reset is showing
if "confirm_reset_rewards" not in st.session_state:
    st.session_state.confirm_reset_rewards = False

# ReWA AI Buddy answers, one per image: {uid: advice_text}
if "buddy_advice" not in st.session_state or not isinstance(
    st.session_state.buddy_advice, dict
):
    st.session_state.buddy_advice = {}

# The cinematic intro plays once per browser session (first run only).
# FIX: the hero's "wait for the intro" delay must apply ONLY on that first
# run. Before, it stayed True for every rerun, so after every click the hero
# text stayed invisible for ~4 seconds and the page felt frozen.
show_intro = PLAY_INTRO and not st.session_state.get("intro_played", False)
if show_intro:
    st.session_state.intro_played = True
hero_after_intro = show_intro

# ---------- THEME (CSS ONLY: Streamlit strips <script> from markdown) ----------
THEME_CSS = """
<style>
                                    398                                                                                 @import
url('https://fonts.googleapis.com/css2?family=Fraunces:opsz,wght@9..144,500;9..144,700;9..144,800&family=Manrope:wg
ht@500;700;800&family=Plus+Jakarta+Sans:wght@400;500;600;700&display=swap');

/* ======================================================================
   DESIGN TOKENS
   ====================================================================== */
:root {
    --deep: #06241A;
    --emerald: #0C4A34;
    --forest: #14432F;
    --leaf: #2E7D52;
    --fern: #4E9A68;
    --sage: #9DB89B;
    --sage-soft: #DDE8D8;
    --mist: #EEF3E8;
    --cream: #F7F3E7;
    --paper: #FCFAF3;
    --gold: #C9A24B;
    --gold-soft: #F1E3BC;
    --ink: #12241A;
    --muted: #566B5E;
    --ease: cubic-bezier(.2, .8, .2, 1);
    --font-display: 'Fraunces', 'Playfair Display', Georgia, serif;
    --font-body: 'Plus Jakarta Sans', 'Inter', system-ui, -apple-system, 'Segoe UI', sans-serif;
}

html,
[data-testid="stMain"],
section.main {
    scroll-behavior: smooth;
    scroll-padding-top: 110px;
}

/* ======================================================================
   PAGE
   ====================================================================== */
.stApp {
    background:
        radial-gradient(circle at 6% 0%, rgba(157, 184, 155, 0.35), transparent 42%),
        radial-gradient(circle at 100% 22%, rgba(46, 125, 82, 0.10), transparent 46%),
        radial-gradient(circle at 20% 100%, rgba(201, 162, 75, 0.10), transparent 40%),
        var(--cream);
    color: var(--ink);
    font-family: var(--font-body);
}

/* subtle paper / leaf-grain texture over the page background
   (pointer-events: none so it can never block a tap or click) */
```

```
.stApp::before {
    content: "";
    position: fixed;
    inset: 0;
    pointer-events: none;
    opacity: 0.07;
          background-image: url("data:image/svg+xml;utf8,<svg xmlns='http://www.w3.org/2000/svg' width='180' height='180'><filter id='n'><feTurbulence type='fractalNoise' baseFrequency='.85' numOctaves='2' stitchTiles='stitch'/><feColorMatrix values='0 0 0 0 0.08 0 0 0 0 0.26 0 0 0 0 0.18 0 0 0 0.9 0'/></filter><rect width='100%' height='100%' filter='url(%23n)'/></svg>");
}

header[data-testid="stHeader"] {
    background: transparent !important;
}

[data-testid="stDecoration"] {
    display: none;
}

.block-container {
    position: relative;
    z-index: 1;
    padding: 5.6rem 2.5rem 3rem 2.5rem;
    max-width: 1320px;
}

.stApp p,
.stApp label,
.stApp li,
.stMarkdown,
.stCaption {
    color: var(--ink);
    font-family: var(--font-body);
}

h1, h2, h3, h4 {
    font-family: var(--font-display) !important;
    color: var(--forest) !important;
    font-weight: 700 !important;
    letter-spacing: -0.01em;
}

hr {
    border: none !important;
    height: 1px !important;
        background: linear-gradient(90deg, transparent, var(--gold), var(--sage), var(--gold), transparent) !important;
    margin: 2.4rem 0 !important;
}

:focus-visible {
    outline: 3px solid var(--gold) !important;
    outline-offset: 2px;
}

/* ======================================================================
   SECTION HEADINGS
   ====================================================================== */
.anchor {
    display: block;
    height: 0;
    scroll-margin-top: 120px;
}

.sec-head {
    margin: 0.2rem 0 0.4rem 0;
}

.sec-eyebrow {
    display: inline-flex;
    align-items: center;
    gap: 12px;
    font-family: var(--font-body);
    font-weight: 700;
```

```
    font-size: 0.72rem;
    letter-spacing: 0.3em;
    text-transform: uppercase;
    color: #1F6A43;
}

.sec-eyebrow::before {
    content: "";
    width: 32px;
    height: 1px;
    background: linear-gradient(90deg, transparent, var(--gold));
}

.stApp .sec-head .sec-title {
    font-size: clamp(1.7rem, 3.1vw, 2.5rem) !important;
    line-height: 1.1 !important;
    margin: 0.3rem 0 0 0 !important;
    padding: 0 !important;
}

.stApp .sec-head h1.sec-title {
    font-size: clamp(2.1rem, 4.2vw, 3.3rem) !important;
}

/* ======================================================================
   FIXED NAVIGATION (glass pill)
   ====================================================================== */
.topnav {
    position: fixed;
    top: 14px;
    left: 50%;
    z-index: 1000;
    display: flex;
    align-items: center;
    gap: 2px;
    padding: 6px 8px 6px 16px;
    border-radius: 999px;
    background: rgba(6, 36, 26, 0.72);
    -webkit-backdrop-filter: blur(14px) saturate(1.4);
    backdrop-filter: blur(14px) saturate(1.4);
    border: 1px solid rgba(201, 162, 75, 0.45);
    box-shadow: 0 14px 34px rgba(6, 36, 26, 0.30),
                inset 0 1px 0 rgba(255, 255, 255, 0.10);
    animation: navIn 1.1s var(--ease) both;
    pointer-events: auto;
}

.stApp .topnav a {
    position: relative;
    display: inline-flex;
    align-items: center;
    gap: 7px;
    padding: 8px 14px;
    border-radius: 999px;
    color: #E8F4E2 !important;
    font-family: var(--font-body);
    font-weight: 600;
    font-size: 0.8rem;
    letter-spacing: 0.2px;
    text-decoration: none !important;
    transition: background 0.3s var(--ease), color 0.3s var(--ease),
                transform 0.3s var(--ease);
}

.stApp .topnav a:hover {
    background: rgba(255, 255, 255, 0.14);
    color: #D8F8B6 !important;
    transform: translateY(-1px);
}

.stApp .topnav a.brand {
    font-family: var(--font-display);
    font-weight: 800;
    font-size: 0.95rem;
    margin-right: 6px;
```

```
    padding-left: 0;
}

.stApp .topnav a.brand:hover {
    background: transparent;
    transform: none;
}

.topnav .ic {
    font-size: 0.95rem;
}

@keyframes navIn {
    from { opacity: 0; transform: translate(-50%, -34px); }
    to   { opacity: 1; transform: translate(-50%, 0); }
}

@media (max-width: 760px) {
    .block-container { padding: 5.2rem 1rem 2rem 1rem; }
    .topnav { padding: 5px 6px 5px 12px; }
    .stApp .topnav a { padding: 8px 10px; }
    .topnav .tx { display: none; }
    .stApp .topnav a.brand .tx { display: inline; }
}

/* ======================================================================
   AMBIENT FLOATING LEAVES (behind all content)
   FIX: pointer-events none on the whole layer and everything inside it.
   ====================================================================== */
.leaf-field {
    position: fixed;
    inset: 0;
    z-index: -1;
    pointer-events: none;
    overflow: hidden;
}

.leaf-field,
.leaf-field * {
    pointer-events: none !important;
}

.leaf {
    position: absolute;
    width: var(--s);
    height: var(--s);
    color: var(--c);
    pointer-events: none;
}

.leaf svg {
    display: block;
    width: 100%;
    height: 100%;
    fill: currentColor;
}

.leaf-field .leaf {
    top: -10vh;
    left: var(--x);
    opacity: 0;
    will-change: transform, opacity;
    animation: leafDrift var(--d) linear var(--dl) infinite;
}

.leaf-field .leaf svg {
    filter: drop-shadow(0 5px 6px rgba(6, 36, 26, 0.16));
}

@keyframes leafDrift {
    0%   { transform: translate3d(0, -8vh, 0) rotate(0deg); opacity: 0; }
    8%   { opacity: 0.42; }
    25%  { transform: translate3d(5vw, 25vh, 0) rotate(80deg); }
    50%  { transform: translate3d(-4vw, 55vh, 0) rotate(190deg); opacity: 0.38; }
    75%  { transform: translate3d(6vw, 82vh, 0) rotate(280deg); opacity: 0.32; }
```

```
    100% { transform: translate3d(0, 114vh, 0) rotate(370deg); opacity: 0; }
}

/* ================================================================
   CINEMATIC INTRO (plays once per session, ~5 seconds)
   FIX: the intro is a full-screen overlay. It used to keep catching every
   tap/click until its animation finished (and forever if the animation was
   paused or throttled). It now NEVER intercepts pointer events, and when it
   finishes it is also pushed behind everything (z-index -1).
   ================================================================ */
.intro {
    position: fixed;
    inset: 0;
    z-index: 99999;
    overflow: hidden;
    pointer-events: none;
    animation: introGone 0.01s linear 4.95s forwards;
}

.intro,
.intro * {
    pointer-events: none !important;
}

@keyframes introGone {
    to { visibility: hidden; opacity: 0; z-index: -1; }
}

.intro-panel {
    position: absolute;
    top: 0;
    bottom: 0;
    width: 50.6%;
    z-index: 0;
}

.intro-panel.left {
    left: 0;
    background: linear-gradient(90deg, #04180F, #0A3A2A);
    animation: gateL 1.2s cubic-bezier(.77, 0, .18, 1) 3.7s forwards;
}

.intro-panel.right {
    right: 0;
    background: linear-gradient(90deg, #0A3A2A, #04180F);
    animation: gateR 1.2s cubic-bezier(.77, 0, .18, 1) 3.7s forwards;
}

@keyframes gateL { to { transform: translateX(-102%); } }
@keyframes gateR { to { transform: translateX(102%); } }

.intro-seam {
    position: absolute;
    left: 50%;
    top: 0;
    bottom: 0;
    width: 2px;
    margin-left: -1px;
    z-index: 2;
    opacity: 0;
    transform: scaleY(0);
    background: linear-gradient(transparent, var(--gold), transparent);
    animation: seamIn 1s ease 0.2s forwards, seamOut 0.4s ease 3.6s forwards;
}

@keyframes seamIn  { to { opacity: 1; transform: scaleY(1); } }
@keyframes seamOut { to { opacity: 0; } }

.intro-stage {
    position: absolute;
    inset: 0;
    z-index: 1;
    display: flex;
    align-items: center;
    justify-content: center;
```

```
    animation: stageOut 0.7s ease 3.5s forwards;
}

@keyframes stageOut {
    to { opacity: 0; transform: scale(1.1); filter: blur(8px); }
}

.intro-glow {
    position: absolute;
    width: 80vmin;
    height: 80vmin;
    border-radius: 50%;
     background: radial-gradient(circle, rgba(168, 230, 192, 0.55), rgba(46, 125, 82, 0.25) 40%, transparent
70%);
    filter: blur(12px);
    opacity: 0;
    transform: scale(0.4);
    animation: glowIn 2.2s var(--ease) 0.1s forwards,
               glowPulse 2.6s ease-in-out 2.3s infinite;
}

@keyframes glowIn   { to { opacity: 1; transform: scale(1); } }
@keyframes glowPulse {
    0%, 100% { filter: blur(12px) brightness(1); }
    50%      { filter: blur(16px) brightness(1.25); }
}

.intro-rings span {
    position: absolute;
    left: 50%;
    top: 50%;
    width: 22vmin;
    height: 22vmin;
    margin: -11vmin 0 0 -11vmin;
    border-radius: 50%;
    border: 1px solid rgba(201, 162, 75, 0.6);
    opacity: 0;
    animation: ringOut 3s cubic-bezier(.2, .6, .2, 1) infinite;
}

.intro-rings span:nth-child(2) { animation-delay: 0.9s; }
.intro-rings span:nth-child(3) { animation-delay: 1.8s; }

@keyframes ringOut {
    0%   { transform: scale(0.4); opacity: 0.9; }
    100% { transform: scale(3.6); opacity: 0; }
}

.intro-leaves {
    position: absolute;
    inset: 0;
    overflow: hidden;
}

.intro-leaves .leaf {
    bottom: -14%;
    left: var(--x);
    opacity: 0;
    animation: leafRise var(--d) ease-out var(--dl) both;
}

.intro-leaves .leaf svg {
    filter: drop-shadow(0 0 8px rgba(168, 230, 192, 0.75));
}

@keyframes leafRise {
    0%   { transform: translate3d(0, 0, 0) rotate(0deg); opacity: 0; }
    15%  { opacity: 0.95; }
    60%  { transform: translate3d(7vw, -70vh, 0) rotate(170deg); }
    100% { transform: translate3d(-4vw, -125vh, 0) rotate(320deg); opacity: 0; }
}

.intro-core {
    position: relative;
    z-index: 3;
```

```
    display: flex;
    flex-direction: column;
    align-items: center;
    text-align: center;
    padding: 0 18px;
}

.intro-mark {
    width: clamp(76px, 12vmin, 124px);
    animation: markIn 1.3s var(--ease) 0.3s both;
}

.intro-mark .rewa-mark {
    display: block;
    width: 100%;
    height: auto;
    animation: markGlow 2.4s ease-in-out 1.6s infinite;
}

@keyframes markIn {
    from { opacity: 0; transform: scale(0.25) rotate(-140deg); }
    to   { opacity: 1; transform: none; }
}

@keyframes markGlow {
    0%, 100% { filter: drop-shadow(0 0 10px rgba(168, 230, 192, 0.5)); }
    50%      { filter: drop-shadow(0 0 30px rgba(216, 248, 182, 0.95)); }
}

.intro-word {
    margin-top: 10px;
    font-family: 'Manrope', var(--font-body);
    font-weight: 800;
    font-size: clamp(2.8rem, 10vmin, 6rem);
    line-height: 1.05;
    letter-spacing: -1px;
    color: #FFFFFF;
    perspective: 700px;
    white-space: nowrap;
}

.intro-word span {
    display: inline-block;
    opacity: 0;
    transform: translateY(34px) rotateX(-70deg);
    transform-origin: 50% 100%;
    animation: letterIn 0.9s var(--ease) forwards;
    animation-delay: calc(1.05s + var(--i) * 0.09s);
}

.intro-word span.g {
    background: linear-gradient(180deg, #E4FBC2, #9BE08A);
    -webkit-background-clip: text;
    background-clip: text;
    -webkit-text-fill-color: transparent;
}

@keyframes letterIn {
    to { opacity: 1; transform: none; }
}

.intro-sub {
    margin-top: 12px;
    font-family: var(--font-body);
    font-weight: 500;
    font-size: clamp(0.62rem, 1.7vmin, 0.82rem);
    letter-spacing: 0.38em;
    padding-left: 0.38em;
    text-transform: uppercase;
    color: #CFE8D2;
    opacity: 0;
    animation: fadeUp 1s var(--ease) 1.8s forwards;
}

.intro-bar {
```

```
    width: min(240px, 52vw);
    height: 2px;
    margin-top: 24px;
    border-radius: 2px;
    overflow: hidden;
    background: rgba(255, 255, 255, 0.14);
    opacity: 0;
    animation: fadeUp 0.6s ease 1.9s forwards;
}

.intro-bar i {
    display: block;
    width: 100%;
    height: 100%;
    transform-origin: left;
    transform: scaleX(0);
    background: linear-gradient(90deg, var(--gold), #D8F8B6);
    animation: fillBar 1.5s cubic-bezier(.65, 0, .35, 1) 2s forwards;
}

@keyframes fillBar { to { transform: scaleX(1); } }

@keyframes fadeUp {
    from { opacity: 0; translate: 0 14px; }
    to   { opacity: 1; translate: none; }
}

/* ======================================================================
   HERO  (deep forest, glowing orbs, parallax leaves, grain)
   ====================================================================== */
.hero {
    position: relative;
    overflow: hidden;
    isolation: isolate;
    min-height: clamp(440px, 64vh, 620px);
    display: flex;
    align-items: center;
    justify-content: center;
    padding: 56px 110px;
    margin: 0 0 34px 0;
    border-radius: 34px;
    border: 1px solid rgba(201, 162, 75, 0.55);
    background:
        radial-gradient(ellipse 60% 80% at 88% 0%, rgba(157, 184, 155, 0.55), transparent 62%),
        radial-gradient(ellipse 50% 70% at 8% 100%, rgba(78, 154, 104, 0.50), transparent 66%),
        linear-gradient(125deg, #06241A 0%, #0C4A34 46%, #2E7D52 100%);
    box-shadow: 0 34px 64px -22px rgba(6, 36, 26, 0.60),
                inset 0 1px 0 rgba(255, 255, 255, 0.16);
}

.hero::before {
    content: "";
    position: absolute;
    top: 0;
    left: 70px;
    right: 70px;
    height: 2px;
    z-index: 4;
    pointer-events: none;
    background: linear-gradient(90deg, transparent, var(--gold), transparent);
}

.hero::after {
    content: "";
    position: absolute;
    inset: 0;
    z-index: 1;
    pointer-events: none;
    opacity: 0.22;
    mix-blend-mode: overlay;
          background-image:  url("data:image/svg+xml;utf8,<svg  xmlns='http://www.w3.org/2000/svg'  width='200'
height='200'><filter      id='n'><feTurbulence      type='fractalNoise'      baseFrequency='.9'      numOctaves='2'
stitchTiles='stitch'/></filter><rect width='100%' height='100%' filter='url(%23n)'/></svg>");
}
```

```
.hero-bg {
    position: absolute;
    inset: 0;
    z-index: 0;
    pointer-events: none;
}

.hero-bg,
.hero-bg * {
    pointer-events: none !important;
}

.hero-orb {
    position: absolute;
    border-radius: 50%;
    filter: blur(8px);
}

.hero-orb.o1 {
    width: 460px;
    height: 460px;
    right: -90px;
    top: -140px;
    background: radial-gradient(circle, rgba(200, 245, 175, 0.55), transparent 66%);
    animation: orbFloat 14s ease-in-out infinite;
}

.hero-orb.o2 {
    width: 520px;
    height: 520px;
    left: -140px;
    bottom: -220px;
    background: radial-gradient(circle, rgba(120, 200, 140, 0.50), transparent 66%);
    animation: orbFloat 18s ease-in-out -6s infinite reverse;
}

@keyframes orbFloat {
    0%, 100% { transform: translate3d(0, 0, 0) scale(1); }
    50%      { transform: translate3d(-26px, 22px, 0) scale(1.08); }
}

.hero-layer {
    position: absolute;
    inset: 0;
    pointer-events: none;
}

.hero-layer .leaf {
    left: var(--x);
    top: var(--y);
    opacity: 0.5;
    animation: leafFloat var(--d) ease-in-out var(--dl) infinite alternate;
}

.hero-layer.l1 .leaf { filter: blur(1.5px); opacity: 0.42; }

.hero-layer .leaf svg {
    filter: drop-shadow(0 4px 8px rgba(0, 0, 0, 0.25));
}

@keyframes leafFloat {
    from { transform: translate3d(-10px, 8px, 0) rotate(-14deg); }
    to   { transform: translate3d(14px, -16px, 0) rotate(16deg); }
}

.fly {
    position: absolute;
    left: var(--x);
    top: var(--y);
    width: 4px;
    height: 4px;
    border-radius: 50%;
    background: #E4FBC2;
    box-shadow: 0 0 10px 3px rgba(190, 240, 170, 0.7);
    opacity: 0;
```

```
    pointer-events: none;
    animation: twinkle var(--d) ease-in-out var(--dl) infinite;
}

@keyframes twinkle {
    0%, 100% { opacity: 0; transform: scale(0.6); }
    50%      { opacity: 1; transform: scale(1.25) translateY(-8px); }
}

.hero-inner {
    position: relative;
    z-index: 3;
    display: flex;
    flex-direction: column;
    align-items: center;
    text-align: center;
}

/* hero content enters after the intro (or immediately if no intro) */
.hero {
    --base: 0.15s;
}

.hero.after-intro {
    --base: 3.95s;
}

.hero .hero-item {
    animation: heroIn 1.1s var(--ease) both;
    animation-delay: calc(var(--base) + var(--n, 0) * 0.12s);
}

@keyframes heroIn {
    from { opacity: 0; translate: 0 30px; filter: blur(8px); }
    to   { opacity: 1; translate: none; filter: none; }
}

.hero .rewa-mark {
    display: block;
    width: clamp(54px, 5.6vw, 74px);
    height: auto;
    margin-bottom: 6px;
    filter: drop-shadow(0 6px 14px rgba(0, 0, 0, 0.3));
}

.hero-eyebrow {
    font-family: var(--font-body);
    font-weight: 600;
    font-size: 0.72rem;
    letter-spacing: 0.34em;
    padding-left: 0.34em;
    text-transform: uppercase;
    color: var(--gold-soft);
    margin-bottom: 4px;
}

.stApp .hero .wordmark {
    font-family: 'Manrope', var(--font-body);
    font-weight: 800;
    font-size: clamp(2.6rem, 6vw, 4.6rem);
    line-height: 1.04;
    letter-spacing: -1.5px;
    margin: 0;
    color: #FFFFFF !important;
    text-shadow: 0 6px 22px rgba(0, 0, 0, 0.28);
    white-space: nowrap;
}

.stApp .hero .wordmark .g {
    background: linear-gradient(100deg, #B8F27E 20%, #F6FFD8 50%, #B8F27E 80%);
    background-size: 220% 100%;
    -webkit-background-clip: text;
    background-clip: text;
    -webkit-text-fill-color: transparent;
    animation: shimmer 7s linear infinite;
```

```
}

@keyframes shimmer {
    from { background-position: 0% 0; }
    to   { background-position: -220% 0; }
}

.stApp .hero .full-name {
    font-family: 'Manrope', var(--font-body);
    font-weight: 500;
    font-size: clamp(0.92rem, 1.6vw, 1.22rem);
    letter-spacing: 0.3px;
    margin: 8px 0 0 0;
    color: #F0FAEA !important;
}

.hero .rule {
    display: flex;
    align-items: center;
    gap: 12px;
    width: min(320px, 70%);
    margin: 16px 0 14px 0;
}

.hero .rule span.line {
    flex: 1;
    height: 1px;
    background: linear-gradient(90deg, transparent, rgba(232, 250, 220, 0.85));
}

.hero .rule span.line.r {
    background: linear-gradient(270deg, transparent, rgba(232, 250, 220, 0.85));
}

.hero .rule-leaf {
    width: 16px;
    height: 16px;
    flex-shrink: 0;
}

.stApp .hero .tagline {
    font-family: var(--font-body);
    font-weight: 500;
    font-size: clamp(0.64rem, 0.95vw, 0.8rem);
    text-transform: uppercase;
    letter-spacing: 0.42em;
    padding-left: 0.42em;
    margin: 0;
    color: #EAF8DF !important;
}

.hero .tagline .dot {
    margin: 0 0.9em;
    color: #D2F9A0;
}

.cta-row {
    display: flex;
    flex-wrap: wrap;
    justify-content: center;
    gap: 12px;
    margin-top: 28px;
}

.stApp .hero a.cta {
    position: relative;
    overflow: hidden;
    display: inline-flex;
    align-items: center;
    gap: 8px;
    padding: 13px 26px;
    border-radius: 999px;
    font-family: var(--font-body);
    font-weight: 700;
    font-size: 0.9rem;
```

```
    letter-spacing: 0.3px;
    text-decoration: none !important;
    transition: transform 0.4s var(--ease), box-shadow 0.4s var(--ease),
                background 0.4s var(--ease), color 0.4s var(--ease);
}

.stApp .hero a.cta.primary {
    color: var(--deep) !important;
    background: linear-gradient(135deg, #F7F3E7, #D8F8B6);
    box-shadow: 0 10px 26px rgba(0, 0, 0, 0.28), 0 0 0 0 rgba(216, 248, 182, 0.5);
    animation: ctaPulse 3.2s ease-in-out infinite;
}

.stApp .hero a.cta.ghost {
    color: #F0FAEA !important;
    border: 1px solid rgba(240, 250, 234, 0.55);
    background: rgba(255, 255, 255, 0.08);
}

.stApp .hero a.cta:hover {
    transform: translateY(-3px) scale(1.03);
}

.stApp .hero a.cta.primary:hover {
    box-shadow: 0 16px 34px rgba(0, 0, 0, 0.32), 0 0 28px rgba(216, 248, 182, 0.65);
}

.stApp .hero a.cta.ghost:hover {
    background: rgba(255, 255, 255, 0.18);
}

@keyframes ctaPulse {
    0%, 100% { box-shadow: 0 10px 26px rgba(0, 0, 0, 0.28), 0 0 0 0 rgba(216, 248, 182, 0.45); }
    50%      { box-shadow: 0 10px 26px rgba(0, 0, 0, 0.28), 0 0 0 12px rgba(216, 248, 182, 0); }
}

.hero-chips {
    display: flex;
    flex-wrap: wrap;
    justify-content: center;
    gap: 10px;
    margin-top: 22px;
}

.hero-chips .chip {
    padding: 6px 14px;
    border-radius: 999px;
    font-size: 0.78rem;
    font-weight: 600;
    color: #EAF8DF;
    background: rgba(255, 255, 255, 0.10);
    border: 1px solid rgba(236, 251, 226, 0.35);
    -webkit-backdrop-filter: blur(4px);
    backdrop-filter: blur(4px);
}

/* icon tiles (right) */
.hero .tiles {
    position: absolute;
    z-index: 3;
    right: 34px;
    top: 50%;
    transform: translateY(-50%);
    display: flex;
    flex-direction: column;
    gap: 10px;
    pointer-events: none;
}

.hero .tile {
    width: 48px;
    height: 48px;
    display: flex;
    align-items: center;
    justify-content: center;
```

```
    border-radius: 15px;
    border: 1px solid rgba(236, 251, 226, 0.55);
    background: rgba(255, 255, 255, 0.10);
    -webkit-backdrop-filter: blur(4px);
    backdrop-filter: blur(4px);
    box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.14);
    animation: tileBob 5s ease-in-out infinite;
    transition: background 0.3s ease, border-color 0.3s ease, scale 0.3s var(--ease);
}

.hero .tile:nth-child(2) { animation-delay: -1.2s; }
.hero .tile:nth-child(3) { animation-delay: -2.4s; }
.hero .tile:nth-child(4) { animation-delay: -3.6s; }

.hero .tile svg {
    width: 22px;
    height: 22px;
}

@keyframes tileBob {
    0%, 100% { transform: translateY(0); }
    50%      { transform: translateY(-6px); }
}

/* Optional: user-supplied banner picture */
.hero.image-mode {
    padding: 0;
    min-height: 0;
    display: block;
}

.hero img.logo-img {
    display: block;
    width: 100%;
    height: auto;
}

@media (max-width: 1100px) {
    .hero { padding: 52px 80px; }
}

@media (max-width: 900px) {
    .hero .tiles { display: none; }
}

@media (max-width: 760px) {
    .hero {
        padding: 44px 18px;
        min-height: 420px;
        border-radius: 24px;
    }
    .stApp .hero .tagline { letter-spacing: 0.25em; padding-left: 0.25em; }
}

/* ======================================================================
   SCROLL-DRIVEN MOTION (parallax + reveals)
   Works in browsers that support CSS scroll timelines (Chrome, Edge,
   recent Safari). Others get a gentle fade-in instead.

   FIX: scroll-driven "reveal" animations were applied to Streamlit's
   interactive containers (bordered panels, expanders, charts). That kept
   them transformed / faded and could leave buttons, uploaders and the
   camera unresponsive. Reveals are now limited to plain, non-interactive
   markup (section headings, reward cards, metrics).
   ====================================================================== */
@keyframes heroParallax {
    to { translate: 0 var(--py, 80px); }
}

@keyframes heroFade {
    to { opacity: 0.12; translate: 0 56px; scale: 0.97; }
}

@keyframes revealUp {
    from { opacity: 0; translate: 0 42px; scale: 0.985; }
```

```
    to   { opacity: 1; translate: none; scale: none; }
}

@supports (animation-timeline: scroll()) {
    .hero-layer {
        animation: heroParallax linear both;
        animation-timeline: scroll(nearest);
        animation-range: 0px 700px;
    }
    .hero-inner {
        animation: heroFade linear both;
        animation-timeline: scroll(nearest);
        animation-range: 0px 540px;
    }
}

@supports (animation-timeline: view()) {
    .reward-card,
    .sec-head {
        animation: revealUp linear both;
        animation-timeline: view();
        animation-range: entry 0px entry 260px;
    }
}

@supports not (animation-timeline: view()) {
    .reward-card,
    .sec-head {
        animation: revealUp 0.9s var(--ease) both;
    }
}

/* ======================================================================
   BORDERED PANELS (st.container(border=True))
   FIX: no backdrop-filter and no hover-transform on these wrappers. They
   contain buttons, uploaders and the camera; a transform / blur on an
   ancestor moves the target under your finger or cursor and can break
   fixed-position popups (dropdown menus, camera permission prompt).
   ====================================================================== */
div[data-testid="stVerticalBlockBorderWrapper"] {
    background: rgba(255, 255, 255, 0.82);
    border: 1px solid rgba(201, 162, 75, 0.35) !important;
    border-radius: 22px !important;
    padding: 8px 10px;
    box-shadow: 0 12px 30px rgba(20, 67, 47, 0.08);
    transition: box-shadow 0.45s var(--ease), border-color 0.45s var(--ease);
}

div[data-testid="stVerticalBlockBorderWrapper"]:hover {
    border-color: rgba(201, 162, 75, 0.65) !important;
    box-shadow: 0 22px 46px rgba(20, 67, 47, 0.14);
}

/* AI result card */
.result {
    position: relative;
    background: linear-gradient(135deg, var(--sage-soft), #F7FBF4);
    border-radius: 18px;
    padding: 18px 22px;
    border-left: 6px solid var(--gold);
    margin: 10px 0 14px 0;
    transition: transform 0.4s var(--ease), box-shadow 0.4s var(--ease);
}

.result:hover {
    transform: translateX(4px);
    box-shadow: 0 10px 26px rgba(20, 67, 47, 0.12);
}

.result h3 {
    color: var(--forest) !important;
    margin: 0 0 4px 0 !important;
}

.result p {
```

```
    color: var(--ink) !important;
    margin: 0 !important;
}

/* ReWA AI Buddy banner */
.buddy-head {
    position: relative;
    overflow: hidden;
    background: linear-gradient(120deg, var(--deep), var(--forest) 60%, var(--leaf));
    border: 1px solid rgba(201, 162, 75, 0.6);
    border-radius: 18px;
    padding: 16px 22px;
    margin: 18px 0 12px 0;
}

.buddy-head::after {
    content: "";
    position: absolute;
    top: 0;
    bottom: 0;
    width: 40%;
    left: -50%;
    pointer-events: none;
    background: linear-gradient(100deg, transparent, rgba(255, 255, 255, 0.16), transparent);
    animation: sheen 6s ease-in-out infinite;
}

@keyframes sheen {
    0%, 55% { left: -50%; }
    100%    { left: 130%; }
}

.stApp .buddy-head .buddy-title {
    font-family: var(--font-display);
    color: #FFFFFF !important;
    font-size: 1.5rem;
    font-weight: 700;
    margin: 0;
}

.stApp .buddy-head .buddy-sub {
    color: var(--gold-soft) !important;
    font-size: 0.85rem;
    letter-spacing: 0.8px;
    margin: 2px 0 0 0;
}

/* Reward cards */
.reward-card {
    position: relative;
    overflow: hidden;
    background: linear-gradient(180deg, #FFFFFF, var(--paper));
    border: 1px solid rgba(201, 162, 75, 0.35);
    border-top: 4px solid var(--gold);
    border-radius: 20px;
    padding: 20px 18px;
    text-align: center;
    box-shadow: 0 12px 26px rgba(20, 67, 47, 0.08);
    transition: transform 0.45s var(--ease), box-shadow 0.45s var(--ease),
                border-color 0.45s var(--ease);
}

.reward-card::after {
    content: "";
    position: absolute;
    top: 0;
    bottom: 0;
    width: 45%;
    left: -70%;
    pointer-events: none;
    background: linear-gradient(100deg, transparent, rgba(255, 255, 255, 0.7), transparent);
    transition: left 0.8s var(--ease);
}

.reward-card:hover {
```

```
    transform: translateY(-7px);
    border-top-color: var(--leaf);
    box-shadow: 0 24px 44px rgba(20, 67, 47, 0.18);
}

.reward-card:hover::after {
    left: 130%;
}

.reward-card .big {
    font-family: var(--font-display);
    font-size: 2.1rem;
    font-weight: 800;
    color: var(--forest);
    line-height: 1.2;
}

.reward-card .small {
    font-size: 0.8rem;
    font-weight: 600;
    letter-spacing: 1.2px;
    text-transform: uppercase;
    color: #6B5B2A;
}

.badge {
    display: inline-block;
    background: var(--gold-soft);
    color: var(--forest);
    border: 1px solid var(--gold);
    border-radius: 999px;
    padding: 2px 12px;
    font-weight: 700;
    font-size: 0.85rem;
}

/* Streamlit metrics as soft cards */
[data-testid="stMetric"] {
    background: #FFFFFF;
    border: 1px solid rgba(201, 162, 75, 0.35);
    border-radius: 18px;
    padding: 14px 18px;
    box-shadow: 0 8px 18px rgba(20, 67, 47, 0.06);
    transition: transform 0.4s var(--ease), box-shadow 0.4s var(--ease);
}

[data-testid="stMetric"]:hover {
    transform: translateY(-4px);
    box-shadow: 0 16px 30px rgba(20, 67, 47, 0.13);
}

[data-testid="stMetricValue"] {
    font-family: var(--font-display);
    color: var(--forest);
    font-weight: 800;
}

/* ======================================================================
   BUTTONS
   ====================================================================== */
div.stButton > button {
    position: relative;
    overflow: hidden;
    background: linear-gradient(135deg, var(--forest), var(--emerald));
    color: #FFFFFF !important;
    border-radius: 14px;
    border: 1px solid var(--gold);
    min-height: 46px;
    font-family: var(--font-body);
    font-weight: 600;
    letter-spacing: 0.3px;
    box-shadow: 0 8px 18px rgba(6, 36, 26, 0.24);
    touch-action: manipulation;
    transition: transform 0.35s var(--ease), box-shadow 0.35s var(--ease),
                background 0.35s var(--ease);
```

```
}

div.stButton > button::after {
    content: "";
    position: absolute;
    inset: 0;
    background: linear-gradient(110deg, transparent 30%, rgba(255, 255, 255, 0.30) 50%, transparent 70%);
    transform: translateX(-120%);
    transition: transform 0.8s var(--ease);
    pointer-events: none;
}

div.stButton > button p {
    color: #FFFFFF !important;
}

div.stButton > button:hover {
    background: linear-gradient(135deg, var(--leaf), var(--forest));
    border-color: var(--gold);
    color: #FFFFFF !important;
    transform: translateY(-3px);
    box-shadow: 0 16px 30px rgba(46, 125, 82, 0.38);
}

div.stButton > button:hover::after {
    transform: translateX(120%);
}

div.stButton > button:active {
    transform: translateY(0) scale(0.985);
    box-shadow: 0 6px 12px rgba(6, 36, 26, 0.22);
}

div.stButton > button:disabled {
    background: #B9C7BE;
    border-color: #B9C7BE;
    box-shadow: none;
    color: #FFFFFF !important;
    transform: none;
}

div.stButton > button:disabled::after {
    display: none;
}

/* Danger buttons (type="primary"): used for destructive confirmations */
div.stButton > button[kind="primary"],
div.stButton > button[data-testid="stBaseButton-primary"] {
    background: linear-gradient(135deg, #B93B3B, #8E2323);
    border: 1px solid #8E2323;
    box-shadow: 0 8px 18px rgba(142, 35, 35, 0.30);
}

div.stButton > button[kind="primary"]:hover,
div.stButton > button[data-testid="stBaseButton-primary"]:hover {
    background: linear-gradient(135deg, #D64545, #A52A2A);
    border-color: #A52A2A;
    box-shadow: 0 16px 28px rgba(165, 42, 42, 0.38);
}

.reset-panel {
    background: linear-gradient(135deg, #FFF6F4, #FFFFFF);
    border: 1px solid rgba(185, 59, 59, 0.35);
    border-left: 6px solid #B93B3B;
    border-radius: 18px;
    padding: 16px 20px;
    margin: 10px 0 12px 0;
}

.reset-panel .reset-title {
    font-family: var(--font-display);
    font-size: 1.15rem;
    font-weight: 700;
    color: #8E2323;
    margin: 0 0 4px 0;
```

```
}

.reset-panel .reset-text {
    font-size: 0.92rem;
    color: var(--ink);
    margin: 0;
}

/* On touch screens there is no real hover: don't leave buttons "stuck"
   lifted after a tap. */
@media (hover: none) {
    div.stButton > button:hover {
        transform: none;
    }
    .reward-card:hover,
    [data-testid="stMetric"]:hover {
        transform: none;
    }
}

/* ======================================================================
   INPUTS: tabs, uploader, camera, expanders, alerts
   ====================================================================== */
.stTabs [data-baseweb="tab-list"] {
    gap: 6px;
    border-bottom: 1px solid rgba(201, 162, 75, 0.45);
}

.stTabs [data-baseweb="tab"] {
    font-weight: 600;
    color: var(--forest);
    border-radius: 12px 12px 0 0;
    padding: 8px 16px;
    transition: background 0.3s ease;
}

.stTabs [data-baseweb="tab"]:hover {
    background: rgba(157, 184, 155, 0.25);
}

.stTabs [aria-selected="true"] {
    background: var(--sage-soft);
    color: var(--forest);
}

.stTabs [data-baseweb="tab-highlight"] {
    background-color: var(--gold);
}

[data-testid="stCameraInput"] {
    border-radius: 18px;
    overflow: hidden;
}

[data-testid="stCameraInput"] button {
    border-radius: 12px;
}

[data-testid="stFileUploader"] {
    border-radius: 16px;
}

[data-testid="stFileUploaderDropzone"] {
    background: linear-gradient(135deg, #FFFFFF, var(--mist));
    border: 2px dashed rgba(46, 125, 82, 0.45);
    border-radius: 18px;
    transition: border-color 0.35s ease, box-shadow 0.35s ease,
                background 0.35s ease;
}

[data-testid="stFileUploaderDropzone"]:hover {
    border-color: var(--gold);
    background: linear-gradient(135deg, #FFFFFF, var(--sage-soft));
    box-shadow: 0 0 0 6px rgba(201, 162, 75, 0.14);
}
```

```
1713
1714  [data-testid="stFileUploader"] label,
1715  [data-testid="stFileUploader"] small {
1716      color: var(--ink) !important;
1717  }
1718
1719  .batch-pill {
1720      display: inline-block;
1721      background: var(--gold-soft);
1722      color: var(--forest);
1723      border: 1px solid var(--gold);
1724      border-radius: 999px;
1725      padding: 3px 14px;
1726      font-weight: 700;
1727      font-size: 0.85rem;
1728      margin: 4px 0 8px 0;
1729  }
1730
1731  [data-testid="stExpander"] {
1732      border: 1px solid rgba(201, 162, 75, 0.35) !important;
1733      border-radius: 18px !important;
1734      background: rgba(255, 255, 255, 0.72);
1735      margin-bottom: 12px;
1736      transition: box-shadow 0.4s var(--ease), border-color 0.4s var(--ease);
1737  }
1738
1739  [data-testid="stExpander"]:hover {
1740      border-color: rgba(201, 162, 75, 0.7) !important;
1741      box-shadow: 0 14px 30px rgba(20, 67, 47, 0.10);
1742  }
1743
1744  [data-testid="stAlert"] {
1745      border-radius: 14px;
1746  }
1747
1748  [data-testid="stImage"] img {
1749      border-radius: 18px;
1750      box-shadow: 0 10px 24px rgba(11, 46, 34, 0.18);
1751      transition: transform 0.5s var(--ease), box-shadow 0.5s var(--ease);
1752  }
1753
1754  [data-testid="stImage"] img:hover {
1755      transform: scale(1.025);
1756      box-shadow: 0 18px 36px rgba(11, 46, 34, 0.26);
1757  }
1758
1759  /* ======================================================================
1760     PROGRESS BAR (daily goal)
1761     ====================================================================== */
1762  .pbar-wrap {
1763      margin: 6px 0 14px 0;
1764  }
1765
1766  .pbar-label {
1767      font-size: 0.95rem;
1768      color: var(--ink);
1769      margin-bottom: 6px;
1770  }
1771
1772  .pbar-track {
1773      width: 100%;
1774      height: 12px;
1775      background: rgba(20, 67, 47, 0.07);
1776      border: 1px solid rgba(20, 67, 47, 0.10);
1777      border-radius: 999px;
1778      overflow: hidden;
1779  }
1780
1781  .pbar-fill {
1782      position: relative;
1783      height: 100%;
1784      background: linear-gradient(90deg, var(--forest), var(--fern));
1785      border-radius: 999px;
1786      transform-origin: left;
1787      animation: barGrow 1.3s var(--ease) both;
```

```
    transition: width 0.5s var(--ease);
}

.pbar-fill::after {
    content: "";
    position: absolute;
    inset: 0;
    pointer-events: none;
    background: linear-gradient(100deg, transparent 30%, rgba(255, 255, 255, 0.45) 50%, transparent 70%);
    background-size: 200% 100%;
    animation: barSheen 3s linear infinite;
}

@keyframes barGrow  { from { transform: scaleX(0); } to { transform: scaleX(1); } }
@keyframes barSheen { from { background-position: 200% 0; } to { background-position: -100% 0; } }

/* ========================================================================
   VIRTUAL BIN CARDS (accent colour per bin via CSS variables)
   ======================================================================== */
.bin-card {
    padding: 4px 4px 2px 4px;
}

.bin-head {
    display: flex;
    align-items: center;
    gap: 14px;
    background: linear-gradient(135deg, var(--tint), rgba(255, 255, 255, 0));
    border: 1px solid var(--line);
    border-radius: 18px;
    padding: 14px 16px;
}

.bin-medal {
    width: 54px;
    height: 54px;
    flex-shrink: 0;
    border-radius: 50%;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 1.6rem;
    background: #FFFFFF;
    border: 2px solid var(--accent);
    box-shadow: 0 0 0 4px var(--tint), 0 6px 14px rgba(11, 46, 34, 0.12);
    transition: transform 0.5s var(--ease);
}

.bin-card:hover .bin-medal {
    transform: rotate(-10deg) scale(1.1);
}

.bin-name {
    font-family: var(--font-display);
    font-size: 1.45rem;
    font-weight: 700;
    color: var(--forest);
    line-height: 1.1;
}

.bin-id {
    font-size: 0.7rem;
    letter-spacing: 2px;
    text-transform: uppercase;
    color: #6B5B2A;
    margin-top: 5px;
}

.bin-count-row {
    display: flex;
    align-items: baseline;
    gap: 10px;
    margin: 20px 4px 16px 4px;
}

```

```
.bin-count {
    font-family: var(--font-display);
    font-size: 2.9rem;
    font-weight: 800;
    color: var(--ink);
    line-height: 1;
}

.bin-count-label {
    font-size: 0.74rem;
    letter-spacing: 1.8px;
    text-transform: uppercase;
    color: #5B6B60;
}

.bin-cap-row {
    display: flex;
    justify-content: space-between;
    margin: 0 4px 7px 4px;
    font-size: 0.74rem;
    letter-spacing: 1.8px;
    text-transform: uppercase;
    color: #5B6B60;
}

.bin-cap-val {
    font-weight: 700;
    color: var(--ink);
}

.bin-track {
    height: 10px;
    margin: 0 4px;
    background: rgba(20, 67, 47, 0.07);
    border: 1px solid rgba(20, 67, 47, 0.10);
    border-radius: 999px;
    overflow: hidden;
}

.bin-fill {
    height: 100%;
    border-radius: 999px;
    background: linear-gradient(90deg, var(--accent), var(--soft));
    transform-origin: left;
    animation: barGrow 1.2s var(--ease) both;
    transition: width 0.5s var(--ease);
}

.bin-status {
    display: inline-flex;
    align-items: center;
    gap: 8px;
    margin: 18px 4px 10px 4px;
    padding: 5px 14px;
    border-radius: 999px;
    font-size: 0.82rem;
    font-weight: 600;
    letter-spacing: 0.3px;
}

.bin-status .dot {
    width: 8px;
    height: 8px;
    border-radius: 50%;
    background: currentColor;
}

.bin-status.ok   { background: #E3F3E8; color: #1F6F49; }
.bin-status.warn { background: #FBF0CF; color: #8A6A12; }
.bin-status.full { background: #FBE1E1; color: #9B2C2C; }

.bin-status.ok .dot   { animation: statusPulse 2.4s ease-in-out infinite; }
.bin-status.warn .dot { animation: statusPulse 1.6s ease-in-out infinite; }
.bin-status.full .dot { animation: statusPulse 0.9s ease-in-out infinite; }

```

```
@keyframes statusPulse {
    0%, 100% { box-shadow: 0 0 0 0 currentColor; opacity: 1; }
    50%      { box-shadow: 0 0 0 5px transparent; opacity: 0.55; }
}

/* ========================================================================
   FOOTER
   ======================================================================== */
.footer {
    position: relative;
    overflow: hidden;
    text-align: center;
    padding: 38px 26px;
    margin-top: 34px;
    border-radius: 28px;
    border: 1px solid rgba(201, 162, 75, 0.5);
    background:
        radial-gradient(ellipse 60% 100% at 85% 0%, rgba(157, 184, 155, 0.35), transparent 60%),
        linear-gradient(120deg, var(--deep), var(--emerald));
    font-family: var(--font-display);
    letter-spacing: 1px;
}

.stApp .footer,
.stApp .footer * {
    color: #E4F1DF !important;
}

.footer .footer-leaf {
    display: inline-block;
    font-size: 1.6rem;
    margin-bottom: 6px;
    animation: tileBob 4s ease-in-out infinite;
}

.footer .footer-sub {
    font-family: var(--font-body);
    font-size: 0.9rem;
    letter-spacing: 0.4px;
    opacity: 0.85;
}

/* ========================================================================
   ACCESSIBILITY: respect "reduce motion"
   ======================================================================== */
@media (prefers-reduced-motion: reduce) {
    *,
    *::before,
    *::after {
        animation: none !important;
        transition: none !important;
        scroll-behavior: auto !important;
    }
    .intro,
    .leaf-field {
        display: none !important;
    }
    /* with animations off, hero items must still be visible */
    .hero .hero-item {
        opacity: 1 !important;
        translate: none !important;
        filter: none !important;
    }
}
/* ========================================================================
   HOMEPAGE EXPLAINER
   ======================================================================== */
.explain {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 16px;
    margin: 0 0 28px 0;
}

.explain-card {
```

```
    background: #FFFFFF;
    border: 1px solid rgba(201, 162, 75, 0.35);
    border-top: 4px solid var(--leaf);
    border-radius: 18px;
    padding: 18px 20px;
    box-shadow: 0 8px 18px rgba(20, 67, 47, 0.06);
}

.explain-card .ex-label {
    font-size: 0.72rem;
    font-weight: 700;
    letter-spacing: 0.22em;
    text-transform: uppercase;
    color: #1F6A43;
}

.explain-card .ex-title {
    font-family: var(--font-display);
    font-size: 1.2rem;
    font-weight: 700;
    color: var(--forest);
    margin: 4px 0 6px 0;
}

.explain-card p {
    margin: 0;
    font-size: 0.93rem;
    line-height: 1.5;
}

.explain-note {
    margin: -14px 0 26px 0;
    font-size: 0.85rem;
    color: var(--muted);
}

@media (max-width: 860px) {
    .explain { grid-template-columns: 1fr; }
}

/* ======================================================================
   ECO JOURNEY: badges, garden, passport
   ====================================================================== */
.badge-tile {
    background: #FFFFFF;
    border: 1px solid rgba(201, 162, 75, 0.45);
    border-radius: 16px;
    padding: 14px 10px;
    text-align: center;
    min-height: 150px;
}

.badge-tile.locked {
    background: var(--mist);
    border-style: dashed;
}

.badge-tile .badge-icon {
    font-size: 2rem;
    line-height: 1.2;
}

.badge-tile.locked .badge-icon {
    filter: grayscale(1);
    opacity: 0.6;
}

.badge-tile .badge-title {
    font-weight: 700;
    color: var(--forest);
    margin-top: 4px;
}

.badge-tile .badge-desc {
    font-size: 0.8rem;
```

```
    color: var(--ink);
    margin-top: 2px;
}

.badge-tile .badge-state {
    font-size: 0.75rem;
    font-weight: 700;
    color: #1F6A43;
    margin-top: 6px;
}

.garden {
    text-align: center;
    padding: 18px 12px;
    border-radius: 18px;
    background: linear-gradient(180deg, var(--mist), #FFFFFF);
    border: 1px solid rgba(201, 162, 75, 0.35);
    margin-bottom: 10px;
}

.garden .garden-emoji {
    font-size: 3.4rem;
    line-height: 1.2;
}

.garden .garden-name {
    font-family: var(--font-display);
    font-size: 1.3rem;
    font-weight: 700;
    color: var(--forest);
}

.stamp {
    text-align: center;
    padding: 14px 8px;
    border-radius: 16px;
    border: 2px dashed #B9C7BE;
    background: var(--mist);
    margin-bottom: 10px;
}

.stamp.on {
    border: 2px solid var(--accent);
    background: #FFFFFF;
}

.stamp .stamp-icon {
    font-size: 1.8rem;
}

.stamp:not(.on) .stamp-icon {
    filter: grayscale(1);
    opacity: 0.55;
}

.stamp .stamp-name {
    font-weight: 700;
    color: var(--forest);
}

.stamp .stamp-state {
    font-size: 0.75rem;
    color: var(--ink);
}
</style>
"""

# Adds the Log in / account panel styles (defined in auth.py)
THEME_CSS += "\n" + auth.CORNER_CSS

# The Log in button's position and click-fixes now live ONLY in
# auth.CORNER_CSS (one place). Here we just keep the centred nav pill clear
# of the button on narrow screens.
THEME_CSS += """
<style>
```

```
@media (max-width: 1000px) {
    .topnav { top: 66px; }
    .block-container { padding-top: 8.4rem !important; }
}
</style>
"""

NAV_HTML = (
    '<div class="topnav" role="navigation" aria-label="Page sections">'
    '<a class="brand" href="#top"><span class="ic">??</span><span class="tx">ReWA.AI</span></a>'
    '<a href="#upload"><span class="ic">?</span><span class="tx">Classify</span></a>'
    '<a href="#rewards"><span class="ic">?</span><span class="tx">Rewards</span></a>'
    '<a href="#station"><span class="ic">??</span><span class="tx">Station</span></a>'
    '<a href="#analytics"><span class="ic">?</span><span class="tx">Analytics</span></a>'
    '<a href="#bins"><span class="ic">?</span><span class="tx">Bins</span></a>'
    '</div>'
)

AMBIENT_LEAVES_HTML = (
    '<div class="leaf-field" aria-hidden="true">'
    + leaf_field(14, seed=11, size=(18, 46), duration=(20, 38), delay=(-34, 0))
    + '</div>'
)


def build_intro_html():
    """The one-time cinematic opening (pure CSS, auto-dismisses itself)."""
    letters = "".join(
        f'<span class="{"g" if i >= 3 else ""}" style="--i:{i}">{ch}</span>'
        for i, ch in enumerate("ReWA.AI")
    )
    return (
        '<div class="intro" role="presentation" aria-hidden="true">'
        '<div class="intro-panel left"></div>'
        '<div class="intro-panel right"></div>'
        '<div class="intro-seam"></div>'
        '<div class="intro-stage">'
        '<div class="intro-glow"></div>'
        '<div class="intro-rings"><span></span><span></span><span></span></div>'
        '<div class="intro-leaves">'
        + leaf_field(
            16, seed=5, size=(16, 40), duration=(3.4, 5.0),
            delay=(0.0, 1.9), colors=INTRO_LEAF_COLORS
        )
        + fireflies(14, seed=3)
        + '</div>'
        '<div class="intro-core">'
        f'<div class="intro-mark">{INTRO_MARK_SVG}</div>'
        f'<div class="intro-word">{letters}</div>'
        '<div class="intro-sub">Recycling-Reusable Waste Assistant</div>'
        '<div class="intro-bar"><i></i></div>'
        '</div>'
        '</div>'
        '</div>'
    )


def build_hero_html(after_intro, bin_count):
    """The hero section: parallax leaves, glowing orbs, wordmark and CTAs."""
    if USE_BANNER_IMAGE:
        banner_mime, banner_b64 = load_banner_base64()
        if banner_b64:
            return (
                '<div class="hero image-mode" id="top">'
                f'<img class="logo-img" src="data:{banner_mime}base64,{banner_b64}" '
                'alt="ReWA.AI - Recycling-Reusable Waste Assistant">'
                '</div>'
            )

    hero_class = "hero after-intro" if after_intro else "hero"

    # NOTE: the HTML is deliberately written without indentation or blank
    # lines; Streamlit's markdown would otherwise treat it as a code block.
    return (
        f'<div class="{hero_class}" id="top">'
```

```
        '<div class="hero-bg" aria-hidden="true">'
        '<div class="hero-orb o1"></div>'
        '<div class="hero-orb o2"></div>'
        '<div class="hero-layer l1" style="--py:70px">'
        + leaf_field(6, seed=21, size=(46, 86), duration=(7, 12),
                     delay=(-8, 0), colors=HERO_LEAF_COLORS, spread_y=True)
        + '</div>'
        '<div class="hero-layer l2" style="--py:150px">'
        + leaf_field(8, seed=33, size=(20, 40), duration=(5, 9),
                     delay=(-6, 0), colors=HERO_LEAF_COLORS, spread_y=True)
        + fireflies(16, seed=8)
        + '</div>'
        '</div>'
        f'{TILES_HTML}'
        '<div class="hero-inner">'
        f'<div class="hero-item" style="--n:0">{LOGO_MARK_SVG}</div>'
        '<div class="hero-item hero-eyebrow" style="--n:1">Recycle * Reuse * Reward</div>'
        '<div class="hero-item wordmark" style="--n:2">ReW<span class="g">A.AI</span></div>'
        '<div class="hero-item full-name" style="--n:3">(Recycling-Reusable Waste Assistant)</div>'
        '<div class="hero-item rule" style="--n:4">'
        '<span class="line"></span>'
        f'{DIVIDER_LEAF_SVG}'
        '<span class="line r"></span>'
        '</div>'
                  '<div  class="hero-item  tagline"  style="--n:5">Sort  Today<span  class="dot">*</span>Save
Tomorrow</div>'
        '<div class="hero-item cta-row" style="--n:6">'
        '<a class="cta primary" href="#upload">? Start sorting</a>'
        '<a class="cta ghost" href="#rewards">? See your impact</a>'
        '</div>'
        '<div class="hero-item hero-chips" style="--n:7">'
        '<span class="chip">? CLIP vision AI</span>'
        f'<span class="chip">?? {bin_count} smart bins</span>'
        f'<span class="chip">? {POINTS_PER_EVENT} virtual Eco Points per item</span>'
        '</div>'
        '</div>'
        '</div>'
    )


def section_head(eyebrow, title, anchor=None, tag="h2"):
    """A designed section heading with a small eyebrow label."""
    anchor_html = f'<div id="{anchor}" class="anchor"></div>' if anchor else ""
    st.markdown(
        f'{anchor_html}'
        '<div class="sec-head">'
        f'<div class="sec-eyebrow">{eyebrow}</div>'
        f'<{tag} class="sec-title">{title}</{tag}>'
        '</div>',
        unsafe_allow_html=True
    )


# ---------- PROGRESS BAR HELPER ----------
def progress_bar(percent, label=None):
    """Return HTML for a progress bar: dark green fill, soft empty track."""
    percent = max(0.0, min(float(percent), 100.0))
    label_html = f'<div class="pbar-label">{label}</div>' if label else ""
    return (
        '<div class="pbar-wrap">'
        f'{label_html}'
        '<div class="pbar-track">'
        f'<div class="pbar-fill" style="width:{percent}%"></div>'
        '</div>'
        '</div>'
    )


# ---------- BIN CARD HELPER ----------
def bin_card_html(name, data):
    """Return the HTML for one elegant bin card (no buttons)."""
    accent = BIN_COLORS.get(name, FOREST)
    icon = BIN_ICONS.get(name, "??")
    capacity = max(0.0, min(float(data["capacity_percent"]), 100.0))
    count = data["item_count"]
```

```
    if capacity >= 90:
        status_class, status_text = "full", "Nearly full"
    elif capacity >= 70:
        status_class, status_text = "warn", "Filling up"
    else:
        status_class, status_text = "ok", "Available"

    style = (
        f"--accent:{accent};--tint:{accent}1F;"
        f"--line:{accent}55;--soft:{accent}B3;"
    )

    # Written without indentation or blank lines on purpose (markdown rule).
    return (
        f'<div class="bin-card" style="{style}">'
        '<div class="bin-head">'
        f'<div class="bin-medal">{icon}</div>'
        '<div>'
        f'<div class="bin-name">{name} Bin</div>'
        f'<div class="bin-id">{data["bin_id"]}</div>'
        '</div>'
        '</div>'
        '<div class="bin-count-row">'
        f'<div class="bin-count">{count}</div>'
        '<div class="bin-count-label">items recorded</div>'
        '</div>'
        '<div class="bin-cap-row">'
        '<span>Capacity</span>'
        f'<span class="bin-cap-val">{capacity:.0f}%</span>'
        '</div>'
        '<div class="bin-track">'
        f'<div class="bin-fill" style="width:{capacity}%"></div>'
        '</div>'
        f'<div class="bin-status {status_class}">'
        f'<span class="dot"></span>{status_text}'
        '</div>'
        '</div>'
    )


# ---------- DATABASE HELPERS ----------
def _existing_db_paths():
    """Every SQLite file this app might be using (rewards + recycling events)."""
    candidates = []

    for module in (rewards_module, data_base):
        for attr in ("DB_NAME", "DB_PATH", "DATABASE"):
            value = getattr(module, attr, None)
            if value and str(value) not in candidates:
                candidates.append(str(value))

    if "recycling.db" not in candidates:
        candidates.append("recycling.db")

    return [path for path in candidates if os.path.exists(path)]


def _table_columns(cur, table):
    return [col[1] for col in cur.execute(f'PRAGMA table_info("{table}")')]


# ---------- CLEAR RECYCLING EVENTS ----------
def clear_all_events():
    """Delete only the signed-in user's recycling events.

    Eco Points and streaks are left untouched.
    """
    data_base.clear_events()


# ---------- CLEAR ECO POINTS / REWARDS ----------
def clear_all_rewards():
    """Permanently delete all Eco Points, streaks and rewarded events.

```

```
    The recycling-events table (the one with 'item_name' and 'material'
    columns) is left alone, so the virtual bins and the
    "Recent Recycling Events" list are NOT affected.

    1. If rewards.py has its own reset function, that is used.
    2. Otherwise every table that is not the recycling-events table is emptied.
    """
    # 1) Prefer a reset function provided by the rewards module itself
    for func_name in (
        "reset_rewards",
        "clear_rewards",
        "clear_all_rewards",
        "reset_all",
        "reset_points",
    ):
        reset_func = getattr(rewards_module, func_name, None)
        if callable(reset_func):
            reset_func()
            return

    # 2) Fallback: empty the rewards tables directly
    db_paths = _existing_db_paths()
    if not db_paths:
        raise FileNotFoundError("No database file was found.")

    cleared_tables = 0

    for db_path in db_paths:
        conn = sqlite3.connect(db_path)
        try:
            cur = conn.cursor()
            tables = [
                row[0]
                for row in cur.execute(
                    "SELECT name FROM sqlite_master "
                    "WHERE type='table' AND name NOT LIKE 'sqlite_%'"
                )
            ]
            for table in tables:
                columns = _table_columns(cur, table)
                # Skip the recycling-events table
                if "item_name" in columns and "material" in columns:
                    continue
                cur.execute(f'DELETE FROM "{table}"')
                cleared_tables += 1
            conn.commit()
        finally:
            conn.close()

    if cleared_tables == 0:
        raise RuntimeError("No rewards tables were found to clear.")

    # Make sure the rewards tables exist and are ready for new points
    init_rewards_db()


# ---------- CHART HELPERS ----------
def bin_color_scale():
    return alt.Scale(
        domain=list(BIN_COLORS.keys()),
        range=list(BIN_COLORS.values())
    )


def colorful_bar_chart(data, y_title, height=300):
    """Bar chart with one colour per bin. `data` is {bin_name: value}."""
    df = pd.DataFrame(
        {"Category": list(data.keys()), "Value": list(data.values())}
    )
    chart = (
        alt.Chart(df)
        .mark_bar(cornerRadiusTopLeft=6, cornerRadiusTopRight=6)
        .encode(
            x=alt.X("Category:N", sort=None, title=None,
                    axis=alt.Axis(labelAngle=0)),
```

```
            y=alt.Y("Value:Q", title=y_title),
            color=alt.Color("Category:N", scale=bin_color_scale(),
                            legend=None),
            tooltip=[
                alt.Tooltip("Category:N"),
                alt.Tooltip("Value:Q", title=y_title)
            ],
        )
        .properties(height=height)
    )
    st.altair_chart(chart, use_container_width=True)


def donut_chart(rows, value_key, height=300):
    """Donut chart. `rows` is a list of dicts with 'category' and value_key."""
    df = pd.DataFrame(rows)
    chart = (
        alt.Chart(df)
        .mark_arc(innerRadius=70, outerRadius=120)
        .encode(
            theta=alt.Theta(f"{value_key}:Q"),
            color=alt.Color("category:N", scale=bin_color_scale(),
                            legend=alt.Legend(title="Category")),
            tooltip=[
                alt.Tooltip("category:N", title="Category"),
                alt.Tooltip(f"{value_key}:Q", title=value_key.title()),
            ],
        )
        .properties(height=height)
    )
    st.altair_chart(chart, use_container_width=True)


def daily_chart(history, value_key, color, y_title, height=300):
    """Last-7-days bar chart in a single colour."""
    df = pd.DataFrame(history)
    chart = (
        alt.Chart(df)
        .mark_bar(color=color, cornerRadiusTopLeft=6, cornerRadiusTopRight=6)
        .encode(
            x=alt.X("date:N", sort=None, title=None,
                    axis=alt.Axis(labelAngle=0)),
            y=alt.Y(f"{value_key}:Q", title=y_title),
            tooltip=[
                alt.Tooltip("date:N", title="Day"),
                alt.Tooltip(f"{value_key}:Q", title=y_title),
            ],
        )
        .properties(height=height)
    )
    st.altair_chart(chart, use_container_width=True)


# ---------- AI MODEL ----------

# The large CLIP model (~1.7 GB) exceeds Streamlit Community Cloud's memory
# limit and crashes the app, so the smaller base model is the default.
# Set REWA_MODEL (env var or top-level secret) to override.
def _clip_model_name():
    name = os.environ.get("REWA_MODEL")
    if not name:
        try:
            name = st.secrets.get("REWA_MODEL")
        except Exception:
            name = None
    return name or "openai/clip-vit-base-patch32"


CLIP_MODEL = _clip_model_name()


@st.cache_resource(show_spinner="Loading the AI model (first run can take a minute)...")
def load_classifier():
    return pipeline(
        "zero-shot-image-classification",
```

```
        model=CLIP_MODEL,
    )


# ---------- IMAGE HELPERS ----------
def open_image(data):
    """Turn raw bytes into a safe RGB PIL image.

    Raises ValueError (with a message fit for visitors) when the file is
    empty, too big, the wrong format or not really an image.
    """
    return rewa_logic.load_validated_image(data)


def collect_images():
    """Gather every image the user has supplied, from files and from the camera.

    Returns (images, problems). Each image is a dict: {name, image}.
    Each problem is a (file name, reason) pair.
    """
    images = []
    problems = []

    # 1) Files picked in the uploader (can be many)
    for uploaded in st.session_state.get(
        f"uploader_{st.session_state.uploader_key}"
    ) or []:
        try:
            images.append({
                "name": uploaded.name,
                "image": open_image(uploaded.getvalue()),
            })
        except ValueError as error:
            problems.append((uploaded.name, str(error)))
        except Exception as error:
            logger.warning("Unexpected upload error for %s: %s", uploaded.name, error)
            problems.append((uploaded.name, "The file could not be read."))

    # 2) Photos taken live with the camera
    for shot in st.session_state.camera_shots:
        try:
            images.append({
                "name": shot["name"],
                "image": open_image(shot["data"]),
            })
        except ValueError as error:
            problems.append((shot["name"], str(error)))
        except Exception as error:
            logger.warning("Unexpected camera error for %s: %s", shot["name"], error)
            problems.append((shot["name"], "The photo could not be read."))

    return images, problems


# ---------- RECYCLING GUIDANCE ----------
def get_recycling_advice(label):
    # Labels from the model use spaces ("cardboard box"); match on underscores
    label = normalize_label(label)

    # Sharps and medical waste need their own, stricter advice
    if any(word in label for word in SHARPS_KEYWORDS):
        return {
            "item": "Possible sharps or medical waste",
            "material": (
                "Needles, syringes and other medical waste are biohazards. "
                "They can injure people and spread infection, so they must "
                "never go in ordinary recycling or general waste."
            ),
            "prepare": [
                "Do not touch sharps with bare hands, and never recap, "
                "bend or break needles.",
                "Place them in a puncture-resistant sharps container "
                "(or a thick, rigid, sealable container if none is available).",
                "Keep the container upright, closed and out of reach of "
                "children and pets."
```

```
            ],
            "dispose": [
                "Hand the sealed container to a hospital, clinic, pharmacy "
                "or authorised biomedical-waste service.",
                "Never put loose sharps in the regular bin or recycling.",
                "If someone is pricked or cut, wash the area with soap and "
                "water and seek medical advice promptly."
            ],
            "reuse": [
                "Sharps and used medical items must not be reused.",
                "Ask your clinic or pharmacy about a sharps take-back "
                "programme."
            ]
        }

    # Other hazardous items
    if any(word in label for word in HAZARDOUS_KEYWORDS):
        return {
            "item": "Possible hazardous item",
            "material": (
                "Batteries, bulbs, medicines, chemicals, aerosols and paints "
                "need special handling and should not go in ordinary "
                "recycling or general waste."
            ),
            "prepare": [
                "Keep the item intact; do not crush, puncture or open it.",
                "Store it in a dry place away from heat and children.",
                "Keep it in its original container and do not mix "
                "different chemicals together."
            ],
            "dispose": [
                "Use a designated hazardous-waste or take-back collection point.",
                "Never place it in the regular bin or pour it down the drain.",
                "Ask your local waste service if you are unsure."
            ],
            "reuse": [
                "Rechargeable batteries can be reused many times.",
                "Unused medicines and chemicals should go to a take-back "
                "programme, not be passed on."
            ]
        }

    elif "food" in label or "peel" in label:
        return {
            "item": "Possible biodegradable item",
            "material": (
                "Food and plant waste can usually be composted, "
                "but cooked food or meat may have separate rules."
            ),
            "prepare": [
                "Remove any packaging, stickers or plastic.",
                "Drain excess liquid.",
                "Keep it separate from non-biodegradable waste."
            ],
            "dispose": [
                "Place it in a compost or organic-waste bin.",
                "Check local rules for cooked food and meat or dairy.",
                "Do not put compostable waste in recycling bins."
            ],
            "reuse": [
                "Compost it at home for garden use.",
                "Some peels and scraps can be reused in cooking "
                "or as plant fertiliser."
            ]
        }

    elif "plastic_bottle" in label or "plastic_cover" in label:
        return {
            "item": "Possible bottle or cover",
            "material": (
                "Not confirmed. Check the recycling symbol "
                "or material label on the item."
            ),
            "prepare": [
                "Empty any remaining liquid.",
```

```
                "Rinse if needed and let the container drain.",
                "Check whether the cap and label need separate sorting."
            ],
            "dispose": [
                "Check whether your local service accepts this type "
                "of container.",
                "Follow the instructions on your local recycling bins.",
                "If the material is unknown, ask your local waste service."
            ],
            "reuse": [
                "If the container is designed for reuse, clean it "
                "and reuse it as intended.",
                "Avoid reusing disposable containers for purposes "
                "they were not designed for."
            ]
        }

    elif "metal_can" in label or "tin" in label:
        return {
            "item": "Possible metal can",
            "material": (
                "Could be metal, but verify the item before sorting."
            ),
            "prepare": [
                "Empty the contents.",
                "Rinse away leftover food or drink when practical.",
                "Handle sharp edges carefully."
            ],
            "dispose": [
                "Check whether your local service accepts metal cans.",
                "Follow local instructions for placing cans in bins.",
                "Do not place hazardous chemical containers in "
                "ordinary recycling."
            ],
            "reuse": [
                "Consider reuse only if the container is safe "
                "and has no sharp edges.",
                "Keep food-contact reuse limited to containers "
                "designed for that purpose."
            ]
        }

    elif "paper" in label or "cardboard_box" in label:
        return {
            "item": "Possible paper-based item",
            "material": (
                "Paper, coated paper and composite cartons may "
                "need different recycling routes."
            ),
            "prepare": [
                "Keep paper clean and dry.",
                "Remove non-paper parts when practical and required.",
                "Flatten cardboard boxes to save space."
            ],
            "dispose": [
                "Check whether coated or laminated paper is accepted.",
                "Check whether cartons have a separate collection route.",
                "Keep heavily soiled paper out of paper recycling "
                "unless local guidance says otherwise."
            ],
            "reuse": [
                "Reuse clean paper for notes or crafts.",
                "Reuse sturdy cardboard boxes for storage."
            ]
        }

    elif "plastic" in label:
        return {
            "item": "Possible plastic item",
            "material": (
                "The plastic resin type is unknown. Look for "
                "the recycling code or material marking."
            ),
            "prepare": [
                "Empty the item first.",
```

```
                "Rinse it if required by your local collection rules.",
                "Check whether lids or other parts need separate sorting."
            ],
            "dispose": [
                "Do not assume every plastic type is accepted.",
                "Check the local rules for the specific plastic code.",
                "Use a designated collection point if required."
            ],
            "reuse": [
                "Reuse durable items when safe and practical.",
                "Avoid reusing packaging for food or drink unless "
                "it is intended for that purpose."
            ]
        }

    elif "glass" in label or "jar" in label:
        return {
            "item": "Possible glass item",
            "material": (
                "Verify whether it is a recyclable glass container "
                "or another type of glass."
            ),
            "prepare": [
                "Empty the container.",
                "Rinse it if needed.",
                "Handle broken glass carefully; never place loose "
                "shards in a bin without following local instructions."
            ],
            "dispose": [
                "Check whether your local service accepts glass.",
                "Follow local rules for colour-separated glass, if any.",
                "Do not assume mirrors, ceramics or heat-resistant "
                "glass belong in bottle-and-jar recycling."
            ],
            "reuse": [
                "Reuse intact jars or containers when suitable.",
                "Do not reuse chipped or cracked glass containers."
            ]
        }

    else:
        return {
            "item": "Item not confidently categorized",
            "material": (
                "The image prediction does not establish the "
                "item's material."
            ),
            "prepare": [
                "Inspect the item and its material label.",
                "Look for a recycling symbol or product markings.",
                "Avoid guessing when the material is unclear."
            ],
            "dispose": [
                "Use a manual material selection if available.",
                "Check your local waste sorting guidelines.",
                "Use a designated collection point if the item "
                "requires special handling."
            ],
            "reuse": [
                "Consider reuse if the item is safe and functional.",
                "Check whether a suitable repair or take-back "
                "option exists."
            ]
        }


# ---------- RECYCLING BIN HELPERS ----------
def _award_in_database(uid, label, category):
    """Save Eco Points. Returns False if this event was already rewarded."""
    return award_event(event_key=uid, item_label=label, bin_name=category)


def record_and_reward(item, category):
    """Record one analysed image in its recycling bin and award its Eco Points.

```

```
    Logged-in users: the real work lives in rewa_logic.record_and_reward
    (which has unit tests). It returns a (status, detail) tuple.

    Guests: the item is added to the virtual bin for this visit only. Nothing
    is saved in the database and no Eco Points are given (status "guest_ok").
    """
    if is_guest:
        uid = item["uid"]
        fingerprint = item.get("fingerprint")
        if uid in st.session_state.recorded_uids:
            return "duplicate", None
        if fingerprint and fingerprint in st.session_state.recorded_fingerprints:
            return "same_photo", None
        added = record_item(
            category,
            item["best_label"],
            st.session_state.digital_twin_bins,
            persist=False,
        )
        if not added:
            return "full", None
        st.session_state.recorded_uids.add(uid)
        st.session_state.recorded_bins[uid] = category
        if fingerprint:
            st.session_state.recorded_fingerprints.add(fingerprint)
        return "guest_ok", None

    return rewa_logic.record_and_reward(
        item["uid"],
        item["best_label"],
        category,
        recorded_uids=st.session_state.recorded_uids,
        recorded_bins=st.session_state.recorded_bins,
        pending_rewards=st.session_state.pending_rewards,
        record_fn=lambda cat, label: record_item(
            cat, label, st.session_state.digital_twin_bins
        ),
        award_fn=_award_in_database,
        fingerprint=item.get("fingerprint"),
        recorded_fingerprints=st.session_state.recorded_fingerprints,
    )


def status_message(status, detail, label, category):
    """Turn a record_and_reward status into a (kind, text) message."""
    if detail:
        logger.warning("record_and_reward %s: %s", status, detail)
    if status == "login":
        return "warning", (
            "Please log in (Log in panel at the top) to save items and earn "
            "Eco Points."
        )
    if status == "guest_ok":
        return "success", (
            f"? {label} added to the {category} bin for this visit. "
            "As a guest nothing is saved and no Eco Points are earned: "
            "log in (Log in panel at the top) to save items and earn points."
        )
    if status == "ok":
        return "success", (
            f"? +{POINTS_PER_EVENT} Eco Points! {label} recorded in the "
            f"{category} bin."
        )
    if status == "rewarded":
        return "warning", "This recycling event was already rewarded."
    if status == "points_err":
        return "warning", (
            f"{label} was recorded, but its Eco Points could not be saved "
            "right now. ReWA will retry automatically."
        )
    if status == "duplicate":
        return "warning", "This item has already been recorded."
    if status == "same_photo":
        return "warning", (
            "This exact photo was already recorded during this visit, so it "
```

```
            "was not counted twice. Use a new photo for a new item."
        )
    if status == "full":
        return "warning", (
            f"Could not add the item to the {category} bin. It may be full "
            "(simulated: empty it below) or the database may be busy. "
            "Please try again."
        )
    return "warning", "Could not record the item. Please try again."


def suggested_category(item, bin_names):
    """Best-guess bin for an analysed image."""
    suggested = resolve_bin(item["best_label"])
    return suggested if suggested in bin_names else bin_names[0]


# ---------- ONE ANALYSED IMAGE (result card) ----------
def render_result(item, number, total, bin_status):
    """Show the full AI analysis, ReWA Buddy and action plan for one image."""
    best_label = item["best_label"]
    best_score = item["best_score"]
    uid = item["uid"]

    title = f"{number}. {item['name']} ? {best_label.title()} ({best_score:.0%})"

    with st.expander(title, expanded=(total == 1 or number == 1)):
        # Wide layout: photo on the left, verdict on the right
        photo_col, verdict_col = st.columns([1, 2], gap="large")

        with photo_col:
            st.image(
                item["image"],
                caption=item["name"],
                use_container_width=True
            )

        with verdict_col:
            if best_score < 0.55:
                st.warning(
                    "The model score is low. Try a clearer photo "
                    "or verify the item manually."
                )
            else:
                st.success(
                    f"Best guess: {best_label.title()} "
                    f"({best_score:.1%} model score)"
                )

            if is_hazardous_label(best_label):
                st.error(
                    "?? This looks like hazardous waste. Handle it with care "
                    "and do not put it in the regular bin."
                )

            st.markdown(
                f'<div class="result">'
                f'<h3>?? {best_label.title()}</h3>'
                f'<p>Model score: {best_score:.1%}</p>'
                f'</div>',
                unsafe_allow_html=True
            )

            st.write("**Other possible predictions**")
            for other in item["results"][1:]:
                st.write(
                    f"* {other['label'].title()} - "
                    f"{other['score']:.1%}"
                )

        # ---- ReWA AI Buddy ----
        st.markdown(
            '<div class="buddy-head">'
            '<div class="buddy-title">? ReWA AI Buddy</div>'
            '<p class="buddy-sub">RECYCLING &amp;amp; REUSE ADVISOR</p>'
```

```
            '</div>',
            unsafe_allow_html=True
        )

        if st.button(
            "? Ask ReWA for Recycling & Reuse Advice",
            use_container_width=True,
            key=f"buddy_btn_{uid}"
        ):
            try:
                with st.spinner("ReWA is preparing your recycling advice..."):
                    st.session_state.buddy_advice[uid] = generate_recycling_advice(
                        waste_type=best_label,
                        confidence=best_score,
                        bin_status=bin_status
                    )
            except Exception as error:
                st.session_state.buddy_advice.pop(uid, None)
                st.warning(
                    "ReWA AI Buddy is temporarily unavailable. "
                    "Your general recycling guidance is still available."
                )
                show_error_details(error)

        if st.session_state.buddy_advice.get(uid):
            with st.container(border=True):
                st.markdown(st.session_state.buddy_advice[uid])

        # ---- Action plan ----
        advice = get_recycling_advice(best_label)

        st.markdown("## ?? Your Recycling Action Plan")
        st.write(f"**Item assessment:** {advice['item']}")

        st.markdown("### ? Material check")
        st.write(advice["material"])

        st.markdown("### ? Step 1: Prepare the item")
        for index, step in enumerate(advice["prepare"]):
            st.checkbox(
                step,
                key=f"prepare_{uid}_{index}"
            )

        st.markdown("### ?? Step 2: Dispose responsibly")
        for step in advice["dispose"]:
            st.markdown(f"- {step}")

        st.markdown("### ? Step 3: Reuse ideas")
        for step in advice["reuse"]:
            st.markdown(f"- {step}")

        st.info(
            "These are general suggestions based on the AI's predicted "
            "label. Verify the actual material and follow local recycling "
            "rules before disposal."
        )


# ---------- ECO JOURNEY (missions, badges, garden, passport) ----------
def render_eco_extras(rewards):
    """Daily missions, badges, a virtual eco garden and an Eco Passport.

    Everything here is derived from the saved rewards summary and points
    table, so it always agrees with the Eco Points shown above. Nothing is
    stored separately and nothing is invented.
    """
    try:
        points_rows = get_points_by_category() or []
    except Exception as error:
        logger.warning("Could not read points by category: %s", error)
        points_rows = []

    bin_names_now = list(st.session_state.digital_twin_bins.keys())
    advice_count = sum(
```

```
        1 for text in st.session_state.buddy_advice.values() if text
    )
    bins_used = len(set(st.session_state.recorded_bins.values()))

    st.markdown("#### ? Your Eco Journey")
    tab_missions, tab_badges, tab_garden, tab_passport = st.tabs(
        ["? Daily missions", "? Badges", "? Eco garden", "? Eco Passport"]
    )

    with tab_missions:
        missions = rewa_logic.daily_missions(
            rewards["today_events"], DAILY_GOAL, bins_used, advice_count
        )
        for mission in missions:
            percent = int(mission["current"] / mission["target"] * 100)
            mark = "?" if mission["done"] else "?"
            st.markdown(
                progress_bar(
                    percent,
                    f"{mark} {mission['title']} - "
                    f"{mission['current']}/{mission['target']} "
                    f"({mission['scope']})"
                ),
                unsafe_allow_html=True
            )
        st.caption(
            "Missions marked "this session" only count activity from this "
            "visit. Missions give no extra points."
        )

    with tab_badges:
        badges = rewa_logic.badge_progress(rewards, points_rows)
        earned_count = sum(1 for badge in badges if badge["earned"])
        st.caption(f"{earned_count} of {len(badges)} virtual badges earned.")
        for start in range(0, len(badges), 4):
            for col, badge in zip(st.columns(4), badges[start:start + 4]):
                state = "earned" if badge["earned"] else "locked"
                label = (
                    "Earned ?" if badge["earned"]
                    else f"{badge['current']}/{badge['target']}"
                )
                with col:
                    st.markdown(
                        f'<div class="badge-tile {state}">'
                        f'<div class="badge-icon">{badge["icon"]}</div>'
                        f'<div class="badge-title">{badge["title"]}</div>'
                        f'<div class="badge-desc">{badge["description"]}</div>'
                        f'<div class="badge-state">{label}</div>'
                        '</div>',
                        unsafe_allow_html=True
                    )

    with tab_garden:
        stage = rewa_logic.garden_stage(rewards["total_events"])
        st.markdown(
            '<div class="garden">'
            f'<div class="garden-emoji">{stage["emoji"]}</div>'
            f'<div class="garden-name">{stage["name"]}</div>'
            '</div>',
            unsafe_allow_html=True
        )
        if stage["next_name"]:
            st.markdown(
                progress_bar(
                    int(stage["progress"] * 100),
                    f"{stage['remaining']} more rewarded item(s) until "
                    f""{stage['next_name']}""
                ),
                unsafe_allow_html=True
            )
        else:
            st.caption("Your garden is fully grown!")
        st.caption(
            "Your garden grows with every rewarded item. It is a virtual "
            "reward, not a measure of real-world environmental impact."
```

```
        )

    with tab_passport:
        stamps = rewa_logic.passport_stamps(points_rows, bin_names_now)
        got = sum(1 for stamp in stamps if stamp["stamped"])
        st.caption(
            f"{got} of {len(stamps)} bin stamps collected. You earn a stamp "
            "the first time a rewarded item goes into that bin."
        )
        for start in range(0, len(stamps), 3):
            for col, stamp in zip(st.columns(3), stamps[start:start + 3]):
                name = stamp["bin"]
                on = "on" if stamp["stamped"] else ""
                with col:
                    st.markdown(
                        f'<div class="stamp {on}" '
                        f'style="--accent:{BIN_COLORS.get(name, FOREST)}">'
                        f'<div class="stamp-icon">{BIN_ICONS.get(name, "??")}</div>'
                        f'<div class="stamp-name">{name}</div>'
                        f'<div class="stamp-state">'
                        f'{"Stamped" if stamp["stamped"] else "Not yet"}</div>'
                        '</div>',
                        unsafe_allow_html=True
                    )


# ---------- OPENING ANIMATION + THEME + NAV + HERO ----------
# The intro lives in an st.empty() slot so the page layout never shifts
# between the first run and later reruns.
intro_slot = st.empty()
if show_intro:
    # FIX (flash of a giant black leaf): the intro's leaf graphics used to be
    # sent to the browser BEFORE the stylesheet, so for a split second they
    # showed with no styling (huge and black). The CSS now travels in the
    # same message as the intro, so the leaves are styled from the first frame.
    intro_slot.markdown(
        THEME_CSS + "\n" + build_intro_html(),
        unsafe_allow_html=True
    )

# ---------- LOGIN / ACCOUNT PANEL (top of the page) ----------
# A normal block in the page flow (not pinned, not a pop-up), so nothing can
# cover it and it always responds to clicks. See auth.py.
auth.account_corner(current_user)

# NOTE: all HTML below is written without indentation or blank lines
# (outside the <style> block); Streamlit's markdown would otherwise treat
# it as a code block.
st.markdown(
    THEME_CSS
    + "\n"
    + NAV_HTML
    + AMBIENT_LEAVES_HTML
    + build_hero_html(
        hero_after_intro,
        len(st.session_state.digital_twin_bins)
    ),
    unsafe_allow_html=True
)

# One-time message left by the previous run (e.g. after recording an item)
if st.session_state.flash:
    flash_kind, flash_text = st.session_state.flash
    st.session_state.flash = None
    if flash_kind == "success":
        st.success(flash_text)
    else:
        st.warning(flash_text)

if is_guest:
    st.info(
        "? You're browsing as a guest. You can try the classifier and fill "
        "the virtual bins for this visit, but nothing is saved. Log in (panel at "
        "the top) to save your items, earn Eco Points and keep your "
        "streaks."
```

```
    )
elif current_user["provider"] == "public":
    # No login needed: everything works and is saved on the shared visitor
    # account. Records saved before sign-in existed are attached to it
    # automatically, so your old history shows up again.
    try:
        if data_base.count_legacy_events() or rewards_module.count_legacy_rewards():
            data_base.claim_legacy_events(current_user["id"])
            rewards_module.claim_legacy_rewards(current_user["id"])
            rebuild_bins_from_database()
    except Exception as error:
        logger.warning("Could not attach older records: %s", error)
    st.caption(
        "? Everything works without logging in. Your items, Eco Points and "
        "history are saved. (Logging in is optional.)"
    )
else:
    # Records saved before sign-in existed belong to no one yet; let the
    # signed-in person claim them once.
    legacy_total = data_base.count_legacy_events()
    legacy_reward_total = rewards_module.count_legacy_rewards()
    if legacy_total or legacy_reward_total:
        st.info(
            f"Found {legacy_total} recycling event(s) and {legacy_reward_total} "
            "reward(s) saved before sign-in existed."
        )
        if st.button("Add them to my account", key="claim_legacy"):
            data_base.claim_legacy_events(current_user["id"])
            rewards_module.claim_legacy_rewards(current_user["id"])
            rebuild_bins_from_database()
            st.rerun()

# ---------- WHAT IS REWA? (problem -> solution -> action) ----------
st.markdown(
    '<div class="explain">'
    '<div class="explain-card"><div class="ex-label">The problem</div>'
    '<div class="ex-title">Which bin is this?</div>'
    '<p>Many people are unsure where an item belongs, and a wrong guess can '
    'contaminate recycling or put hazardous waste where it should not be.</p>'
    '</div>'
    '<div class="explain-card"><div class="ex-label">The solution</div>'
    '<div class="ex-title">Snap, check, sort</div>'
    '<p>Take a photo. ReWA suggests what the item is, which bin fits, and '
    'how to prepare and reuse it. You always confirm the category.</p>'
    '</div>'
    '<div class="explain-card"><div class="ex-label">Try it now</div>'
    '<div class="ex-title">Three quick steps</div>'
    '<p>1. Add a photo below. 2. Check the suggestion. 3. Add it to your '
    'virtual bin and earn Eco Points. <a href="#upload">Start sorting ?</a></p>'
    '</div>'
    '</div>'
    '<p class="explain-note">AI suggestions can be wrong: always check your '
    'local recycling rules. Bin levels are simulated (no sensors) and Eco '
    'Points are virtual.</p>',
    unsafe_allow_html=True
)

# ---------- ECO REWARDS ----------
# Retry any Eco Points that could not be saved earlier. This is safe: the
# database refuses to reward the same event twice.
if st.session_state.pending_rewards:
    rewa_logic.retry_pending_rewards(
        st.session_state.pending_rewards, _award_in_database
    )

rewards = get_summary()

section_head("Your impact", "? Your Eco Rewards", anchor="rewards")

r1, r2, r3, r4 = st.columns(4)

with r1:
    st.markdown(
        f'<div class="reward-card"><div class="big">? {rewards["total_points"]}</div>'
        f'<div class="small">Eco Points</div></div>',
```

```
        unsafe_allow_html=True
    )

with r2:
    day_word = "day" if rewards["current_streak"] == 1 else "days"
    st.markdown(
        f'<div class="reward-card"><div class="big">? {rewards["current_streak"]} {day_word}</div>'
        f'<div class="small">Current streak</div></div>',
        unsafe_allow_html=True
    )

with r3:
    st.markdown(
        f'<div class="reward-card"><div class="big">? {rewards["longest_streak"]}</div>'
        f'<div class="small">Longest streak (days)</div></div>',
        unsafe_allow_html=True
    )

with r4:
    st.markdown(
        f'<div class="reward-card"><div class="big">? {rewards["active_days"]}</div>'
        f'<div class="small">Active recycling days</div></div>',
        unsafe_allow_html=True
    )

st.write("")

goal_percent = min(int(rewards["today_events"] / DAILY_GOAL * 100), 100)
st.markdown(
    progress_bar(
        goal_percent,
        f"Today's goal: {rewards['today_events']} of {DAILY_GOAL} items "
        f"recycled * {rewards['today_points']} points earned today"
    ),
    unsafe_allow_html=True
)

if rewards["today_events"] == 0 and rewards["current_streak"] > 0:
    st.caption(
        f"Recycle an item today to keep your {rewards['current_streak']}-day "
        f"streak alive!"
    )
elif rewards["today_events"] >= DAILY_GOAL:
    st.caption("? Daily goal complete - great work!")

st.caption(
    f"Each recorded item earns {POINTS_PER_EVENT} virtual Eco Points. They "
    "have no cash value and cannot be redeemed for real-world rewards. "
    "Recording an item in this app does not prove it was physically recycled."
)

# Explain (instead of hiding) any mismatch between items and points
recorded_total = sum(
    data["item_count"] for data in st.session_state.digital_twin_bins.values()
)
points_gap = rewa_logic.reward_gap(recorded_total, rewards["total_events"])

if st.session_state.pending_rewards:
    st.warning(
        f"{len(st.session_state.pending_rewards)} recorded item(s) are "
        "waiting for their Eco Points to be saved. ReWA retries automatically."
    )
elif points_gap:
    st.info(
        f"?? {points_gap} item(s) in your bins have no Eco Points. This "
        "happens for items recorded before rewards existed, or after "
        ""Reset Eco Points" (which clears points but keeps your item "
        "history). Points are never added retroactively, so nothing is "
        "counted twice."
    )

render_eco_extras(rewards)

# ---- Reset Eco Points (two-step confirmation) ----
has_rewards = (
```

```
    rewards["total_points"] > 0
    or rewards["total_events"] > 0
    or rewards["active_days"] > 0
)

if not st.session_state.confirm_reset_rewards:
    # Step 1: the normal button
    reset_btn_col, _reset_space = st.columns([1, 3])
    with reset_btn_col:
        if st.button(
            "?? Reset Eco Points",
            use_container_width=True,
            disabled=not has_rewards,
            key="reset_rewards_start",
            help="Delete all Eco Points and streaks (asks for confirmation first)."
        ):
            st.session_state.confirm_reset_rewards = True
            st.rerun()
else:
    # Step 2: ask once more before anything is deleted
    st.markdown(
        '<div class="reset-panel">'
        '<div class="reset-title">?? Are you absolutely sure?</div>'
        '<p class="reset-text">This will permanently delete '
        f'<b>{rewards["total_points"]} Eco Points</b>, your streaks and your '
        'rewarded-event history. This cannot be undone.</p>'
        '</div>',
        unsafe_allow_html=True
    )

    confirm_col, cancel_col, _confirm_space = st.columns([1.3, 1, 2])

    with confirm_col:
        if st.button(
            "? Yes, delete my Eco Points",
            type="primary",
            use_container_width=True,
            key="reset_rewards_confirm"
        ):
            try:
                clear_all_rewards()
                st.session_state.confirm_reset_rewards = False
                st.session_state.flash = (
                    "success",
                    "? All Eco Points and streaks have been reset."
                )
            except Exception as error:
                st.session_state.confirm_reset_rewards = False
                st.session_state.flash = (
                    "warning",
                    f"Could not reset Eco Points: {error}"
                )
            st.rerun()

    with cancel_col:
        if st.button(
            "?? Cancel",
            use_container_width=True,
            key="reset_rewards_cancel"
        ):
            st.session_state.confirm_reset_rewards = False
            st.rerun()

st.markdown("---")

section_head("Get started", "Welcome to a greener future!", anchor="upload", tag="h1")
st.write(
    "Upload photos or take live pictures of your items and let AI suggest "
    "what they might be. You'll also receive practical recycling guidance "
    "for every item."
)

# Bins are needed by the ReWA AI Buddy inside the analysis cards
bins = st.session_state.digital_twin_bins

```

```
# ---------- MAIN LAYOUT (stacked: full-width upload, then full-width analysis) ----------
upload_row = st.container()
result_row = st.container()

with upload_row:
    with st.container(border=True):
        st.subheader("? 1. Add your images")
        st.write(
            "Upload one or many photos, or snap them live with your camera."
        )

        tab_upload, tab_camera = st.tabs(
            ["? Upload files", "? Take live photo"]
        )

        # ---- Tab 1: upload one or many files ----
        with tab_upload:
            st.file_uploader(
                "Choose photos of your items (you can select several)",
                type=["jpg", "jpeg", "png", "webp"],
                accept_multiple_files=True,
                key=f"uploader_{st.session_state.uploader_key}"
            )

        # ---- Tab 2: live camera ----
        with tab_camera:
            st.caption(
                "Allow camera access in your browser when asked. "
                "Take a picture, then press "Add this photo". "
                "Repeat for as many items as you like."
            )

            camera_photo = st.camera_input(
                "Take a picture of the item",
                key=f"camera_{st.session_state.camera_key}"
            )

            if camera_photo is not None:
                if st.button(
                    "? Add this photo to the batch",
                    use_container_width=True,
                    key="add_camera_photo"
                ):
                    photo_bytes = camera_photo.getvalue()
                    photo_id = hashlib.md5(photo_bytes).hexdigest()

                    if any(s["id"] == photo_id
                           for s in st.session_state.camera_shots):
                        st.info("This photo is already in the batch.")
                    else:
                        st.session_state.camera_shots.append({
                            "id": photo_id,
                            "name": f"Camera photo "
                                    f"{len(st.session_state.camera_shots) + 1}",
                            "data": photo_bytes,
                        })
                        # Fresh camera widget, ready for the next picture
                        st.session_state.camera_key += 1
                        st.session_state.flash = (
                            "success",
                            "? Photo added! Take another one or press "
                            ""Classify" to analyse your batch."
                        )
                        st.rerun()

            if st.session_state.camera_shots:
                st.markdown(
                    f'<span class="batch-pill">? '
                    f'{len(st.session_state.camera_shots)} live photo(s) '
                    f'in the batch</span>',
                    unsafe_allow_html=True
                )
                if st.button(
                    "?? Remove all live photos",
                    use_container_width=True,
```

```
                    key="remove_camera_photos"
                ):
                    st.session_state.camera_shots = []
                    st.session_state.camera_key += 1
                    st.rerun()

        # ---- Everything collected so far ----
        images, problems = collect_images()

        for bad_name, reason in problems:
            st.error(f"Unable to use "{bad_name}". {reason}")

        if len(images) > MAX_IMAGES:
            st.warning(
                f"You added {len(images)} images. Only the first "
                f"{MAX_IMAGES} will be analysed at a time."
            )
            images = images[:MAX_IMAGES]

        if images:
            st.markdown(
                f'<span class="batch-pill">?? {len(images)} image(s) '
                f'ready to classify</span>',
                unsafe_allow_html=True
            )
            st.image(
                [img["image"] for img in images],
                caption=[img["name"] for img in images],
                width=130
            )

        button_left, button_right = st.columns([3, 1])

        with button_left:
            classify_clicked = st.button(
                f"? Classify {len(images)} Image(s)" if images
                else "? Classify Images",
                use_container_width=True,
                disabled=len(images) == 0
            )

        with button_right:
            if st.button(
                "? Clear all",
                use_container_width=True,
                disabled=(len(images) == 0 and not st.session_state.batch)
            ):
                st.session_state.camera_shots = []
                st.session_state.batch = []
                st.session_state.buddy_advice = {}
                st.session_state.uploader_key += 1
                st.session_state.camera_key += 1
                st.rerun()

        st.caption("Supported formats: JPG, PNG and WEBP")

with result_row:
    with st.container(border=True):
        st.subheader("? 2. AI analysis")

        # Run classification only when the button is clicked
        if classify_clicked and images:
            try:
                with st.spinner(
                    "Loading the AI model (the first run can take a minute)..."
                ):
                    classifier = load_classifier()
                new_batch = []
                progress = st.progress(0.0, text="Analyzing your images...")

                for position, item in enumerate(images, start=1):
                    try:
                        predictions = classifier(
                            item["image"],
                            candidate_labels=CANDIDATE_LABELS
```

```
                        )[:3]

                        new_batch.append({
                            # Unique key for this classification; the database
                            # uses it to make sure the same event can never
                            # be rewarded twice.
                            "uid": uuid.uuid4().hex,
                            "name": item["name"],
                            "image": item["image"],
                            "results": predictions,
                            "best_label": predictions[0]["label"],
                            "best_score": predictions[0]["score"],
                            # Identifies the picture itself, so the same photo
                            # cannot be recorded (and rewarded) twice.
                            "fingerprint": hashlib.md5(
                                item["image"].tobytes()
                            ).hexdigest(),
                        })
                    except Exception as error:
                        logger.warning(
                            "Classification failed for %s: %s",
                            item["name"], error
                        )
                        st.error(
                            f"Could not analyze "{item['name']}". "
                            "Try a clearer photo or a different file."
                        )
                        show_error_details(error)

                    progress.progress(
                        position / len(images),
                        text=f"Analyzed {position} of {len(images)} image(s)"
                    )

                progress.empty()

                st.session_state.batch = new_batch
                # New items mean the old ReWA answers no longer apply
                st.session_state.buddy_advice = {}

            except Exception as error:
                st.error(
                    "The AI model could not be loaded. It needs an internet "
                    "connection the first time. Please try again."
                )
                show_error_details(error)

        # ---- Saved results, shown directly under "AI analysis" ----
        batch = st.session_state.batch

        if not batch:
            st.write(
                "Your AI predictions and recycling advice will appear here."
            )
        else:
            if len(batch) > 1:
                st.markdown(
                    f'<span class="batch-pill">? {len(batch)} images '
                    f'analysed</span>',
                    unsafe_allow_html=True
                )

            # Real values from the virtual bins
            bin_status = {
                name.lower(): {
                    "fill_percent": data["capacity_percent"],
                    "source": "simulated"
                }
                for name, data in bins.items()
            }

            for number, item in enumerate(batch, start=1):
                render_result(item, number, len(batch), bin_status)


```

```
# ---------- VIRTUAL RECYCLING STATION (sorting section) ----------
st.markdown("---")
section_head("Sorting", "?? Virtual Recycling Station", anchor="station")

batch = st.session_state.batch
bin_names = list(st.session_state.digital_twin_bins.keys())

if batch:
    pending = [i for i in batch if i["uid"] not in st.session_state.recorded_uids]

    st.write(
        f"**{len(batch)}** item(s) detected * "
        f"**{len(pending)}** waiting to be added."
    )

    # ---- Add everything in one click ----
    if len(batch) > 1:
        if st.button(
            f"? Add all {len(pending)} pending items "
            f"(+{POINTS_PER_EVENT} Eco Points each)",
            disabled=len(pending) == 0,
            use_container_width=True,
            key="add_all_items"
        ):
            points_count = 0
            ok_count = 0
            problems_found = []

            for item in pending:
                # Use the bin currently shown in each item's dropdown
                chosen = st.session_state.get(
                    f"confirmed_category_{item['uid']}",
                    suggested_category(item, bin_names)
                )
                status, detail = record_and_reward(item, chosen)

                if status == "ok":
                    ok_count += 1
                    points_count += 1
                elif status == "guest_ok":
                    ok_count += 1
                elif status == "rewarded":
                    ok_count += 1
                elif status == "points_err":
                    ok_count += 1
                    problems_found.append(
                        f"{item['best_label']}: Eco Points will be retried"
                    )
                else:
                    _, message = status_message(
                        status, detail, item["best_label"], chosen
                    )
                    problems_found.append(f"{item['best_label']}: {message}")

            if problems_found:
                st.session_state.flash = (
                    "warning",
                    f"{ok_count} item(s) recorded. "
                    + " * ".join(problems_found)
                )
            elif is_guest:
                st.session_state.flash = (
                    "success",
                    f"? {ok_count} item(s) added to your bins for this "
                    "visit. Log in (Log in panel at the top) to save items and earn "
                    "Eco Points."
                )
            else:
                st.session_state.flash = (
                    "success",
                    f"? {ok_count} item(s) recorded! "
                    f"+{points_count * POINTS_PER_EVENT} Eco Points."
                )
            st.rerun()

```

```
    # ---- One row per image ----
    for number, item in enumerate(batch, start=1):
        uid = item["uid"]
        best_label = item["best_label"]
        already = uid in st.session_state.recorded_uids

        with st.container(border=True):
            thumb_col, info_col = st.columns([1, 4], gap="medium")

            with thumb_col:
                st.image(item["image"], width=90)

            with info_col:
                st.write(f"**{number}. {item['name']}** - detected: **{best_label}**")

                # Pre-select the bin that matches the detected item
                # (hazardous items always go to the Hazardous bin)
                suggested = resolve_bin(best_label)
                if rewa_logic.needs_manual_category(
                    item["best_score"], suggested
                ):
                    st.warning(
                        "?? The AI is not sure about this item. Please check "
                        "the category below before adding it."
                    )
                default_index = (
                    bin_names.index(suggested) if suggested in bin_names else 0
                )

                category = st.selectbox(
                    "Confirm the waste category",
                    bin_names,
                    index=default_index,
                    key=f"confirmed_category_{uid}",
                    disabled=already
                )

                if already:
                    recorded_bin = st.session_state.recorded_bins.get(uid, category)
                    st.success(
                        f"? Already recorded in the {recorded_bin} recycling bin."
                    )
                elif st.button(
                    f"Add item to recycling bin (+{POINTS_PER_EVENT} Eco Points)",
                    key=f"add_item_{uid}"
                ):
                    status, detail = record_and_reward(item, category)
                    st.session_state.flash = status_message(
                        status, detail, best_label, category
                    )
                    st.rerun()
else:
    st.info("Add photos (upload or live camera) and classify them to get started.")

# ---------- RECYCLING ANALYTICS ----------
st.markdown("---")
section_head("Insights", "? Recycling Analytics", anchor="analytics")

bins = st.session_state.digital_twin_bins

# 1. Calculate the total recorded items
total_items = sum(
    data["item_count"]
    for data in bins.values()
)

# 2. Find the most frequently recorded category
if total_items > 0:
    most_common = max(
        bins,
        key=lambda name: bins[name]["item_count"]
    )
    common_count = bins[most_common]["item_count"]
else:
    most_common = "None yet"
```

```
    common_count = 0

# 3. Calculate average simulated capacity
average_capacity = sum(
    data["capacity_percent"]
    for data in bins.values()
) / len(bins)

# 4. Display summary metrics
col1, col2, col3 = st.columns(3)

with col1:
    st.metric("Total recorded items", total_items)

with col2:
    st.metric("Most recorded category", most_common)

with col3:
    st.metric("Average simulated capacity", f"{average_capacity:.1f}%")

# 5. Compare recorded item counts and simulated capacities side by side
chart_left, chart_right = st.columns(2)

with chart_left:
    st.subheader("Items by Material")
    colorful_bar_chart(
        {name: data["item_count"] for name, data in bins.items()},
        y_title="Items recorded"
    )

with chart_right:
    st.subheader("Virtual Bin Capacity Comparison")
    colorful_bar_chart(
        {name: data["capacity_percent"] for name, data in bins.items()},
        y_title="Simulated capacity (%)"
    )

# 6. Rewards analytics (read from the database, so it survives restarts)
st.subheader("? Rewards Analytics")

# Re-read the summary so points recorded earlier in this run are included
rewards = get_summary()

s1, s2, s3, s4 = st.columns(4)
s1.metric("Total Eco Points", rewards["total_points"])
s2.metric("Rewarded events", rewards["total_events"])
s3.metric("Current streak", f"{rewards['current_streak']} days")
s4.metric("Longest streak", f"{rewards['longest_streak']} days")

points_by_category = get_points_by_category()
history = get_daily_history(days=7)

donut_col, daily_col = st.columns(2)

with donut_col:
    st.markdown("**Points earned by category**")
    if points_by_category:
        donut_chart(points_by_category, value_key="points")
    else:
        st.info("Record an item to see your points by category.")

with daily_col:
    st.markdown("**Daily progress - items recycled (last 7 days)**")
    if rewards["total_events"] > 0:
        daily_chart(
            history,
            value_key="events",
            color="#2E8B57",
            y_title="Items recycled"
        )
    else:
        st.info("Your daily progress will appear after your first item.")

if rewards["total_events"] > 0:
    st.markdown("**Eco Points earned per day (last 7 days)**")
```

```
    daily_chart(
        history,
        value_key="points",
        color="#C9A24B",
        y_title="Eco Points",
        height=240
    )

# ---------- RECENT RECYCLING EVENTS ----------
st.markdown("---")
section_head("Activity", "? Recent Recycling Events")

events = get_events()

if events:
    for event in events[:10]:
        st.write(
            f"?? {event['item_name']} - "
            f"{event['material']} - "
            f"{event['timestamp']}"
        )

    confirm_clear = st.checkbox(
        "Yes, permanently delete all recycling events (Eco Points are kept)",
        key="confirm_clear_events"
    )

    if st.button("? Clear recent events", disabled=not confirm_clear):
        try:
            clear_all_events()
            # Keep the screen consistent with the (now empty) event log.
            # Eco Points are NOT touched; the rewards database still refuses
            # to reward the same event twice if an item is added again.
            rebuild_bins_from_database()
            st.session_state.recorded_uids = set()
            st.session_state.recorded_bins = {}
            st.session_state.recorded_fingerprints = set()
            st.toast("All recycling events cleared!")
            st.rerun()
        except Exception as error:
            st.error(f"Could not clear events: {error}")
else:
    st.info("No recycling events recorded yet.")

# ---------- DISPLAY VIRTUAL BINS ----------
st.markdown("---")
section_head("Digital twin", "? Your Recycling Bins", anchor="bins")
st.caption(
    "Six virtual bins. Item counts come from your recorded events; "
    "capacity is a simulation, not a sensor measurement. "
    "No hardware is connected."
)

bins = st.session_state.digital_twin_bins


def display_bin(name, data):
    with st.container(border=True):
        st.markdown(bin_card_html(name, data), unsafe_allow_html=True)

        if st.button(
            f"?? Empty {name} bin",
            key=f"empty_{name}",
            use_container_width=True
        ):
            empty_bin(name, st.session_state.digital_twin_bins)
            st.rerun()


# Display all bins, three per row
bin_list = list(bins.keys())
for i in range(0, len(bin_list), 3):
    row_cols = st.columns(3, gap="medium")
    for row_col, name in zip(row_cols, bin_list[i:i + 3]):
        with row_col:
```

```
            display_bin(name, bins[name])

# ---------- FOOTER ----------
st.markdown("""
<div class="footer">
<div class="footer-leaf">?</div>
<div>ReWA.AI  *  Reduce waste * Reuse * Recycle</div>
<div class="footer-sub">Right item. Right bin. A cleaner tomorrow.</div>
</div>
""", unsafe_allow_html=True)
```

## auth.py

```
"""auth.py - sign-in for ReWA.AI

Everyone can open the website. A visitor who has not logged in is a "guest"
(they can try the classifier, but nothing is saved for them). A "Log in"
panel sits at the top of the page; once logged in it turns into an account
panel with a Sign out button.

DESIGN NOTE (why it is built this way): the login panel is a normal block in
the page flow (an expander). It is NOT pinned with position:fixed and it is
NOT a pop-up dialog, because those two tricks were what could be covered,
blocked or broken by the page's heavy CSS/animations. A normal block cannot
be covered, so the button always responds.

Ways to log in (each one is switched on by its own block in
.streamlit/secrets.toml):
  1. Google account  -> Streamlit's built-in OpenID Connect  [auth]
  2. Phone number    -> SMS one-time code via Twilio Verify   [twilio]
  3. Demo login      -> LOCAL TESTING ONLY, no verification   [dev]

Usage in app.py:
    import auth
    current_user = auth.resolve_user()     # never blocks; guests are allowed
    data_base.set_current_user(current_user["id"])
    auth.account_corner(current_user)      # Log in panel / account panel

current_user = {
    "id": "google:123" | "phone:+9198..." | "demo:name" | "guest:abc...",
    "name": ..., "provider": ..., "is_guest": True | False,
}
Use current_user["id"] as the key that separates each person's records.

Requires: streamlit>=1.42, Authlib (for Google), requests (for Twilio).
"""
import re
import time
import uuid

import requests
import streamlit as st

# True  -> everyone can use EVERY feature without logging in. Visitors share
#          one saved "public" account (items, Eco Points, streaks, history).
# False -> visitors who are not logged in are guests (nothing is saved).
ALLOW_ANONYMOUS = True
PUBLIC_USER_ID = "public"

OTP_COOLDOWN_SECONDS = 30   # wait between "Send code" presses
OTP_MAX_ATTEMPTS = 5        # wrong codes allowed before a new code is needed
PHONE_RE = re.compile(r"^\+[1-9]\d{7,14}$")  # E.164, e.g. +919876543210
CODE_RE = re.compile(r"^\d{4,10}$")

# Session keys that survive a change of user
_KEEP = {"phone_user", "demo_user", "active_user", "guest_id", "intro_played"}

# Styles for the login panel. app.py appends this to its theme CSS.
# Deliberately plain: no fixed positioning, nothing that can cover a click.
CORNER_CSS = """
<style>
/* Hide Streamlit's own Deploy button */
[data-testid="stAppDeployButton"] { display: none !important; }

.st-key-auth_corner {
    max-width: 460px;
    margin: 0 0 14px 0;
    position: relative;
    z-index: 5;
}

.st-key-auth_corner [data-testid="stExpander"] {
    background: rgba(255, 255, 255, 0.92);
}
```

```
 72
 73  .st-key-auth_corner summary {
 74      font-weight: 700;
 75  }
 76
 77  .st-key-auth_corner div.stButton > button {
 78      pointer-events: auto !important;
 79  }
 80  </style>
 81  """
 82
 83
 84  # ---------- config helpers ----------
 85  def _google_provider():
 86      """Return (enabled, provider_name).
 87
 88      Streamlit supports two secrets layouts:
 89        [auth] client_id = ...           -> default provider (name None)
 90        [auth.google] client_id = ...    -> named provider ("google")
 91      """
 92      if not hasattr(st, "login"):
 93          return False, None
 94      try:
 95          cfg = st.secrets["auth"]
 96          if cfg.get("client_id"):
 97              return True, None
 98          named = []
 99          for name in cfg.keys():
100              sub = cfg[name]
101              if hasattr(sub, "get") and sub.get("client_id"):
102                  named.append(name)
103          if named:
104              return True, ("google" if "google" in named else named[0])
105      except Exception:
106          pass
107      return False, None
108
109
110  def _google_enabled():
111      return _google_provider()[0]
112
113
114  def _twilio():
115      """Return the [twilio] secrets, or None if phone login is not configured."""
116      try:
117          cfg = st.secrets["twilio"]
118          if cfg["account_sid"] and cfg["auth_token"] and cfg["verify_sid"]:
119              return cfg
120      except Exception:
121          pass
122      return None
123
124
125  def _demo_enabled():
126      """Demo login is OFF unless [dev] enabled = true is in secrets.toml."""
127      try:
128          return bool(st.secrets["dev"]["enabled"])
129      except Exception:
130          return False
131
132
133  # ---------- Twilio Verify (plain REST, no extra package) ----------
134  def _send_code(phone):
135      cfg = _twilio()
136      r = requests.post(
137          f"https://verify.twilio.com/v2/Services/{cfg['verify_sid']}/Verifications",
138          data={"To": phone, "Channel": "sms"},
139          auth=(cfg["account_sid"], cfg["auth_token"]),
140          timeout=15,
141      )
142      r.raise_for_status()
143
144
145  def _check_code(phone, code):
146      cfg = _twilio()
```

```
    r = requests.post(
        f"https://verify.twilio.com/v2/Services/{cfg['verify_sid']}/VerificationCheck",
        data={"To": phone, "Code": code},
        auth=(cfg["account_sid"], cfg["auth_token"]),
        timeout=15,
    )
    return r.ok and r.json().get("status") == "approved"


# ---------- current user ----------
def get_user():
    """Return the logged-in user dict, or None if nobody is logged in."""
    if _google_enabled() and getattr(st.user, "is_logged_in", False):
        email = getattr(st.user, "email", None)
        uid = getattr(st.user, "sub", None) or email
        if uid:
            return {
                "id": f"google:{uid}",
                "name": getattr(st.user, "name", None) or email or "Google user",
                "email": email,
                "provider": "google",
            }

    phone = st.session_state.get("phone_user")
    if phone:
        return {"id": f"phone:{phone}", "name": phone, "phone": phone, "provider": "phone"}

    demo = st.session_state.get("demo_user")
    if demo and _demo_enabled():
        return {"id": f"demo:{demo}", "name": demo, "provider": "demo"}

    return None


def resolve_user():
    """Return the current user, or a temporary guest. Never blocks the page."""
    user = get_user()

    if user is None and ALLOW_ANONYMOUS:
        user = {
            "id": PUBLIC_USER_ID,
            "name": "Visitor",
            "provider": "public",
            "is_guest": False,
        }
    elif user is None:
        guest_id = st.session_state.get("guest_id")
        if not guest_id:
            guest_id = uuid.uuid4().hex
            st.session_state.guest_id = guest_id
        user = {
            "id": f"guest:{guest_id}",
            "name": "Guest",
            "provider": "guest",
            "is_guest": True,
        }
    else:
        user["is_guest"] = False

    # A different identity on this browser tab (guest -> user, user -> guest,
    # or user A -> user B): wipe the previous identity's batch, bins, advice
    # etc. so nothing leaks between accounts.
    if st.session_state.get("active_user") != user["id"]:
        for key in list(st.session_state.keys()):
            if key not in _KEEP:
                del st.session_state[key]
        st.session_state.active_user = user["id"]

    return user


# ---------- small display helpers ----------
def _mask_phone(phone):
    phone = phone or ""
    return phone[:3] + "*" * max(len(phone) - 7, 0) + phone[-4:] if len(phone) > 7 else phone
```

```
222
223
224 def _short_name(user):
225     if user["provider"] == "phone":
226         return _mask_phone(user["name"])
227     first = (user.get("name") or "").split("@")[0].split(" ")[0]
228     return first[:14] or "Account"
229
230
231 # ---------- login forms ----------
232 def _start_google_login():
233     """Button callback. Runs BEFORE the page reruns (same as the original
234     working pattern). Any error is saved and shown instead of vanishing."""
235     st.session_state.pop("login_error", None)
236     _, provider = _google_provider()
237     try:
238         if provider:
239             st.login(provider)
240         else:
241             st.login()
242     except Exception as error:
243         st.session_state["login_error"] = str(error)
244
245
246 def _google_form():
247     st.button(
248         "? Continue with Google",
249         on_click=_start_google_login,
250         use_container_width=True,
251         key="g_login",
252     )
253     error = st.session_state.get("login_error")
254     if error:
255         st.error(
256             "Google login is not set up correctly. Check the [auth] block in "
257             ".streamlit/secrets.toml (redirect_uri, cookie_secret, client_id, "
258             "client_secret, server_metadata_url) and that Authlib is "
259             "installed (pip install Authlib)."
260         )
261         st.caption(f"Technical details: {error}")
262
263
264 def _phone_form():
265     st.markdown("**Log in with your phone**")
266
267     raw = st.text_input(
268         "Phone number with country code",
269         placeholder="+919876543210",
270         key="phone_input",
271     )
272     phone = re.sub(r"[\s\-()]", "", raw or "")
273
274     sent_to = st.session_state.get("otp_sent_to")
275     last = st.session_state.get("otp_sent_at", 0.0)
276     wait = int(OTP_COOLDOWN_SECONDS - (time.time() - last))
277
278     if st.button(
279         "Resend code" if sent_to else "Send code",
280         use_container_width=True,
281         disabled=bool(sent_to) and wait > 0,
282         key="otp_send",
283     ):
284         if not PHONE_RE.match(phone):
285             st.error("Enter the number in international format, e.g. +919876543210.")
286         else:
287             sent = False
288             try:
289                 _send_code(phone)
290                 sent = True
291             except Exception:
292                 st.error(
293                     "Could not send the code. Check the number and your "
294                     "Twilio settings (trial accounts can only text numbers "
295                     "verified in Twilio) and try again."
296                 )
```

```
            if sent:
                st.session_state.otp_sent_to = phone
                st.session_state.otp_sent_at = time.time()
                st.session_state.otp_attempts = 0
                st.rerun()

    if sent_to:
        note = f"Code sent to {_mask_phone(sent_to)}."
        if wait > 0:
            note += f" You can ask for a new one in {wait}s."
        st.caption(note)

        code = st.text_input("Verification code", max_chars=10, key="otp_code")

        if st.button("Verify and log in", use_container_width=True, key="otp_verify"):
            attempts = st.session_state.get("otp_attempts", 0)
            code = (code or "").strip()

            if attempts >= OTP_MAX_ATTEMPTS:
                st.error("Too many wrong attempts. Please request a new code.")
            elif not CODE_RE.match(code):
                st.error("Enter the digits from the SMS.")
            else:
                approved = None
                try:
                    approved = _check_code(sent_to, code)
                except Exception:
                    st.error("Could not verify the code right now. Please try again.")

                if approved is True:
                    st.session_state.phone_user = sent_to
                    st.rerun()
                elif approved is False:
                    st.session_state.otp_attempts = attempts + 1
                    st.error("That code is wrong or has expired.")


def _demo_form():
    st.warning(
        "Demo login is for local testing only. It does not verify who you are."
    )
    name = st.text_input("Pick any display name", max_chars=24, key="demo_name")
    if st.button("Continue as demo user", use_container_width=True, key="demo_go"):
        clean = re.sub(r"[^a-z0-9_-]", "", (name or "").strip().lower())
        if len(clean) < 2:
            st.error("Use at least 2 letters or numbers.")
        else:
            st.session_state.demo_user = clean
            st.rerun()


def _login_panel():
    google = _google_enabled()
    phone = _twilio() is not None
    demo = _demo_enabled()

    # Status line: tells you instantly which methods this app can see.
    st.caption(
        f"Streamlit {st.__version__} * "
        f"Google: {'ready' if google else 'not set up'} * "
        f"Phone: {'ready' if phone else 'not set up'} * "
        f"Demo: {'on' if demo else 'off'}"
    )

    if not (google or phone or demo):
        st.error(
            "No login method is set up yet. Create .streamlit/secrets.toml "
            "next to app.py with the Google ([auth]), Twilio ([twilio]) or "
            "demo ([dev]) settings, then fully restart the app."
        )
        return

    st.caption("Log in to save your recycling history, Eco Points and streaks.")

    if google:
```

```
        _google_form()
        if phone or demo:
            st.markdown("**- or -**")

    if phone:
        _phone_form()
        if demo:
            st.markdown("**- or -**")

    if demo:
        _demo_form()


# ---------- the panel ----------
def _sign_out(user):
    st.session_state.pop("phone_user", None)
    st.session_state.pop("demo_user", None)
    st.session_state.pop("active_user", None)
    if user["provider"] == "google":
        st.logout()
    else:
        st.rerun()


def account_corner(user):
    """Login panel for guests, account panel otherwise (a normal page block)."""
    with st.container(key="auth_corner"):
        if user["is_guest"] or user["provider"] == "public":
            label = "? Log in (optional)" if user["provider"] == "public" else "? Log in"
            with st.expander(label, expanded=False):
                _login_panel()
        else:
            with st.expander(f"? {_short_name(user)}", expanded=False):
                label = user.get("email") or (
                    _mask_phone(user["name"]) if user["provider"] == "phone"
                    else user["name"]
                )
                st.markdown(f"**{label}**")
                st.caption(f"Logged in with {user['provider']}")
                if st.button("Sign out", use_container_width=True, key="sign_out"):
                    _sign_out(user)
```

## data_base.py

```
import sqlite3
import threading

DB_NAME = "recycling.db"

# Rows saved before login existed are kept under this id until claimed.
LEGACY_USER = "legacy"

# The signed-in user is stored per THREAD. Streamlit runs each visitor's
# session in its own thread, so two people using the app at the same time
# can never see each other's records (a plain global variable would mix them).
_current = threading.local()


def set_current_user(user_id):
    """Call once at the top of every script run, right after login."""
    _current.user_id = user_id


def _uid():
    user_id = getattr(_current, "user_id", None)
    if not user_id:
        # Fail closed: never fall back to showing everyone's data.
        raise RuntimeError("No signed-in user. Call set_current_user() first.")
    return user_id


def create_data_base():
    connection = sqlite3.connect(DB_NAME)

    connection.execute("""
        CREATE TABLE IF NOT EXISTS recycling_events (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            material TEXT NOT NULL,
            item_name TEXT NOT NULL,
            timestamp TEXT NOT NULL
        )
    """)

    # Migration: add user_id to an existing database (safe to run every start)
    columns = [row[1] for row in connection.execute("PRAGMA table_info(recycling_events)")]
    if "user_id" not in columns:
        connection.execute(
            f"ALTER TABLE recycling_events "
            f"ADD COLUMN user_id TEXT NOT NULL DEFAULT '{LEGACY_USER}'"
        )

    connection.execute(
        "CREATE INDEX IF NOT EXISTS idx_events_user ON recycling_events(user_id)"
    )
    connection.commit()
    connection.close()


def save_event(material, item_name, timestamp):
    connection = sqlite3.connect(DB_NAME)

    connection.execute(
        """
        INSERT INTO recycling_events
        (material, item_name, timestamp, user_id)
        VALUES (?, ?, ?, ?)
        """,
        (material, item_name, timestamp, _uid())
    )

    connection.commit()
    connection.close()


def get_events():
```

```
    connection = sqlite3.connect(DB_NAME)
    connection.row_factory = sqlite3.Row

    rows = connection.execute(
        "SELECT * FROM recycling_events WHERE user_id = ? ORDER BY id DESC",
        (_uid(),)
    ).fetchall()

    connection.close()
    return [dict(row) for row in rows]


def get_bin_totals():
    connection = sqlite3.connect(DB_NAME)

    rows = connection.execute("""
        SELECT material, COUNT(*) AS total
        FROM recycling_events
        WHERE user_id = ?
        GROUP BY material
    """, (_uid(),)).fetchall()

    connection.close()

    return dict(rows)


def clear_events():
    """Delete ONLY the signed-in user's recycling events."""
    connection = sqlite3.connect(DB_NAME)
    connection.execute("DELETE FROM recycling_events WHERE user_id = ?", (_uid(),))
    connection.commit()
    connection.close()


# ---------- one-time adoption of pre-login records ----------
def count_legacy_events():
    connection = sqlite3.connect(DB_NAME)
    total = connection.execute(
        "SELECT COUNT(*) FROM recycling_events WHERE user_id = ?", (LEGACY_USER,)
    ).fetchone()[0]
    connection.close()
    return total


def claim_legacy_events(user_id):
    """Assign all pre-login records to one account."""
    connection = sqlite3.connect(DB_NAME)
    connection.execute(
        "UPDATE recycling_events SET user_id = ? WHERE user_id = ?",
        (user_id, LEGACY_USER)
    )
    connection.commit()
    connection.close()
```

## digital_twin.py

```
import logging
import re
from datetime import datetime

from data_base import create_data_base, save_event

logger = logging.getLogger("rewa")

# 1. Bins (Hazardous + Biodegradable added)
BIN_IDS = {
    "Plastic": "PLASTIC_01",
    "Paper": "PAPER_01",
    "Metal": "METAL_01",
    "Glass": "GLASS_01",
    "Hazardous": "HAZARDOUS_01",
    "Biodegradable": "BIO_01",
}


def create_bins():
    return {
        name: {
            "bin_id": bin_id,
            "item_count": 0,
            "capacity_percent": 0,
            "times_emptied": 0,
            "history": [],
        }
        for name, bin_id in BIN_IDS.items()
    }


bins = create_bins()

# 2. Explicit item -> bin mapping (never defaults to Plastic)
ITEM_TO_BIN = {
    "plastic bottle": "Plastic", "plastic bag": "Plastic",
    "plastic container": "Plastic", "plastic wrapper": "Plastic",
    "paper sheet": "Paper", "newspaper": "Paper",
    "cardboard box": "Paper", "cardboard": "Paper",
    "aluminium can": "Metal", "metal can": "Metal", "tin can": "Metal",
    "glass bottle": "Glass", "glass jar": "Glass",
    "battery": "Hazardous", "medicine": "Hazardous", "light bulb": "Hazardous",
    "syringe or needle": "Hazardous", "medical waste": "Hazardous",
    "biohazard waste": "Hazardous", "chemical container": "Hazardous",
    "aerosol can": "Hazardous", "paint can": "Hazardous",
    "pesticide or poison": "Hazardous",
    "food waste": "Biodegradable", "fruit peel": "Biodegradable",
    "leaves": "Biodegradable",
}

# Fallback keywords, checked in this order (whole words only, so "tin"
# no longer matches inside words like "painting")
KEYWORDS = [
    ("Hazardous", ["battery", "chemical", "paint", "bulb", "medicine",
                   "e-waste", "syringe", "needle", "sharps", "medical",
                   "biohazard", "aerosol", "pesticide", "poison",
                   "hazardous"]),
    ("Biodegradable", ["food", "peel", "leaf", "leaves", "vegetable", "fruit"]),
    ("Glass", ["glass"]),
    ("Metal", ["metal", "aluminium", "aluminum", "tin", "steel", "can"]),
    ("Paper", ["paper", "cardboard", "newspaper", "carton"]),
    ("Plastic", ["plastic", "polythene", "bottle", "bag"]),
]


def _has_word(text, word):
    """True if `word` (or its plural) appears as a whole word in `text`."""
    return re.search(rf"\b{re.escape(word)}s?\b", text) is not None


```

```
def get_bin_for_item(item_name):
    name = item_name.strip().lower()
    if name in ITEM_TO_BIN:
        return ITEM_TO_BIN[name]
    for bin_name, words in KEYWORDS:
        if any(_has_word(name, w) for w in words):
            return bin_name
    return None  # unknown -> NOT Plastic


# 3. Record a new recycling event
def record_item(waste_type, item_name, state=None, persist=True):
    """Add one item to a bin.

    persist=True  -> also save the event in the database (logged-in users).
    persist=False -> bin only (guests: nothing is saved).

    The database write happens FIRST. If it fails, the bin is left
    untouched, so the bins and the saved history can never disagree.
    Returns True on success, False otherwise.
    """
    if state is None:
        state = bins

    # The category chosen by the user wins; item name is only a fallback
    target = waste_type if waste_type in state else get_bin_for_item(item_name)

    if target not in state:
        logger.warning("No bin for '%s' (got '%s'). Not recorded.",
                       item_name, waste_type)
        return False

    selected_bin = state[target]

    if selected_bin["capacity_percent"] >= 100:
        logger.warning("%s bin is full! Please empty it first.", target)
        return False

    now = datetime.now()

    if persist:
        try:
            create_data_base()
            save_event(
                material=target,
                item_name=item_name,
                timestamp=now.isoformat(timespec="seconds"),
            )
        except Exception as error:
            logger.warning("Could not save recycling event: %s", error)
            return False

    selected_bin["item_count"] += 1

    # Simulated capacity, not a real measurement
    selected_bin["capacity_percent"] = min(
        selected_bin["capacity_percent"] + 5, 100
    )

    selected_bin["history"].append({
        "item": item_name,
        "waste_type": target,
        "time": now.strftime("%H:%M:%S"),
    })
    return True


# 4. Empty a bin
def empty_bin(waste_type, state=None):
    if state is None:
        state = bins
    if waste_type not in state:
        logger.warning("Unknown bin: %s", waste_type)
        return False

```

```
    b = state[waste_type]
    b["item_count"] = 0
    b["capacity_percent"] = 0
    b["history"] = []
    b["times_emptied"] += 1
    return True


def empty_all_bins(state=None):
    if state is None:
        state = bins
    for name in state:
        empty_bin(name, state)


# 5. Dashboard (console)
def show_dashboard():
    print("\n===== SMART RECYCLING DIGITAL TWIN =====")
    for waste_type, data in bins.items():
        print(f"\nBin: {waste_type}")
        print(f"ID: {data['bin_id']}")
        print(f"Items: {data['item_count']}")
        print(f"Estimated capacity: {data['capacity_percent']}%")
        print(f"Times emptied: {data['times_emptied']}")


# Run test events only when this file is run directly
if __name__ == "__main__":
    record_item("Plastic", "Plastic bottle")
    record_item("Paper", "Cardboard box")
    record_item("Hazardous", "Battery")
    record_item("Biodegradable", "Fruit peel")
    show_dashboard()
    empty_bin("Paper")
    show_dashboard()
```

## recycling_logic.py

```python
CATEGORY_TO_BIN = {
    "cardboard box": "paper",
    "paper sheet": "paper",
    "newspaper": "paper",
    "plastic bottle": "plastic",
    "plastic bag": "plastic",
    "plastic wrapper": "plastic",
    "metal can": "metal",
    "glass bottle": "glass",
    "glass jar": "glass",
    "food waste": None,    # no bin in the Digital Twin
    "other waste": None,
}

SOFT = "Soft plastic is often NOT accepted in regular plastic bins. Check local rules."
NOTES = {
    "plastic bag": SOFT,
    "plastic wrapper": SOFT,
    "food waste": "No recycling bin in this system. Compost or general waste, per local rules.",
    "other waste": "No matching bin in this system. Check the item's label and local rules.",
}

FULL_AT = 90
LOW_SCORE = 0.55   # same threshold your app uses

def decide_bin(waste_type, score, bins):
    label = waste_type.lower()
    if label not in CATEGORY_TO_BIN:
        return {"bin": None, "fill_percent": None, "is_full": False,
                "needs_verification": True,
                "note": "Unknown category. Verify the item manually."}
    bin_name = CATEGORY_TO_BIN[label]
    fill = bins.get(bin_name, {}).get("fill_percent") if bin_name else None
    return {
        "bin": bin_name,
        "fill_percent": fill,
        "is_full": fill is not None and fill >= FULL_AT,
        "needs_verification": score < LOW_SCORE,
        "note": NOTES.get(label, ""),
    }

def twin_to_shared(twin_bins):
    return {
        name.lower(): {
            "fill_percent": data["capacity_percent"],
            "item_count": data["item_count"],
            "source": "simulated",
        }
        for name, data in twin_bins.items()
    }
```

## requirements.txt

```
--extra-index-url https://download.pytorch.org/whl/cpu
streamlit>=1.42
Authlib
transformers
torch
pillow
pandas
altair
requests
google-genai
```

## rewa_logic.py

```
"""Pure helper logic for ReWA.AI.

No Streamlit imports here on purpose, so everything can be unit-tested
with plain `pytest` (see test_rewa_logic.py).

Everything in this file is DERIVED from real saved data (the rewards
summary and the recycling event log). Nothing here stores or invents
numbers, so badges, garden, missions and points can never disagree with
each other: they all read the same summary.
"""
import io

# ---------------------------------------------------------------------------
# 1. Recording an item and awarding its points (exactly once)
# ---------------------------------------------------------------------------


def _award(uid, label, category, pending_rewards, award_fn):
    """Try to award points. Remember the event for a retry if saving fails."""
    try:
        awarded = award_fn(uid, label, category)
    except Exception as error:  # database locked, file missing, ...
        pending_rewards[uid] = (label, category)
        return "points_err", str(error)
    pending_rewards.pop(uid, None)
    return ("ok" if awarded else "rewarded"), ""


def record_and_reward(uid, label, category, *, recorded_uids, recorded_bins,
                      pending_rewards, record_fn, award_fn,
                      fingerprint=None, recorded_fingerprints=None):
    """Record one confirmed item, then award points for it.

    Status values:
      ok           recorded and rewarded
      rewarded     recorded, database says this event was already rewarded
      points_err   recorded, but points could not be saved (will be retried)
      duplicate    this uid was already recorded -> nothing happens
      same_photo   this exact photo was already recorded this session
      full         the bin refused the item -> nothing recorded, no points
      record_err   recording crashed -> nothing recorded, no points
    """
    if uid in recorded_uids:
        return "duplicate", ""

    if (fingerprint and recorded_fingerprints is not None
            and fingerprint in recorded_fingerprints):
        return "same_photo", ""

    try:
        recorded = record_fn(category, label)
    except Exception as error:
        return "record_err", str(error)

    if not recorded:
        return "full", ""

    recorded_uids.add(uid)
    recorded_bins[uid] = category
    if fingerprint and recorded_fingerprints is not None:
        recorded_fingerprints.add(fingerprint)
    return _award(uid, label, category, pending_rewards, award_fn)


def retry_pending_rewards(pending_rewards, award_fn):
    """Retry points that failed to save earlier. Returns how many succeeded.

    award_fn must be idempotent per uid (the rewards database already
    refuses to reward the same event_key twice), so retrying is safe.
    """
    recovered = 0
```

```
    for uid, (label, category) in list(pending_rewards.items()):
        status, _ = _award(uid, label, category, pending_rewards, award_fn)
        if status in ("ok", "rewarded"):
            recovered += 1
    return recovered


def reward_gap(recorded_items, rewarded_events):
    """How many recorded items have no reward entry (never negative)."""
    return max(int(recorded_items) - int(rewarded_events), 0)


# ---------------------------------------------------------------------------
# 2. Classification uncertainty
# ---------------------------------------------------------------------------

def needs_manual_category(score, suggested_bin, threshold=0.55):
    """True when the user must pick the bin themselves."""
    return suggested_bin is None or score < threshold


# ---------------------------------------------------------------------------
# 3. Image validation
# ---------------------------------------------------------------------------

ALLOWED_FORMATS = ("JPEG", "PNG", "WEBP")
MAX_PIXELS = 40_000_000


def load_validated_image(data, max_bytes=8 * 1024 * 1024, max_side=1600):
    """Return a safe, RGB, size-limited PIL image or raise ValueError.

    The ValueError message is written for end users, so it can be shown.
    """
    from PIL import Image, UnidentifiedImageError

    if not data:
        raise ValueError("The file is empty.")
    if len(data) > max_bytes:
        raise ValueError(f"The file is larger than {max_bytes // (1024 * 1024)} MB.")

    try:
        Image.open(io.BytesIO(data)).verify()  # cheap integrity check
        image = Image.open(io.BytesIO(data))  # verify() invalidates the object
        if image.format not in ALLOWED_FORMATS:
            raise ValueError("Only JPG, PNG and WEBP images are supported.")
        if image.width * image.height > MAX_PIXELS:
            raise ValueError("The image has too many pixels. Please use a smaller photo.")
        image = image.convert("RGB")
    except ValueError:
        raise
    except (UnidentifiedImageError, OSError, SyntaxError, Image.DecompressionBombError) as error:
        raise ValueError("This file is not a readable image.") from error

    if max(image.size) > max_side:
        image.thumbnail((max_side, max_side))  # faster model, same photo
    return image


# ---------------------------------------------------------------------------
# 4. Rewards extras: badges, garden, missions, passport (all derived)
# ---------------------------------------------------------------------------

BADGES = [
    # key, icon, title, description, metric, target
    ("first", "?", "First Sprout", "Record your first item", "events", 1),
    ("five", "?", "Getting Growing", "Record 5 items", "events", 5),
    ("twentyfive", "?", "Eco Regular", "Record 25 items", "events", 25),
    ("streak3", "?", "3-Day Streak", "Record items on 3 days in a row", "streak", 3),
    ("streak7", "?", "7-Day Streak", "Record items on 7 days in a row", "streak", 7),
    ("explorer", "?", "Sorting Explorer", "Earn points in 3 different bins", "bins", 3),
    ("hazard", "??", "Hazard Aware", "Record one hazardous item", "hazard", 1),
]

```

```

def reward_metrics(summary, points_rows):
    rows = points_rows or []
    earned_bins = {r.get("category") for r in rows if (r.get("points") or 0) > 0}
    return {
        "events": int(summary.get("total_events", 0) or 0),
        "streak": int(summary.get("longest_streak", 0) or 0),
        "bins": len(earned_bins),
        "hazard": 1 if "Hazardous" in earned_bins else 0,
    }


def badge_progress(summary, points_rows):
    """List of badges with earned flag and current/target progress."""
    metrics = reward_metrics(summary, points_rows)
    result = []
    for key, icon, title, desc, metric, target in BADGES:
        current = metrics[metric]
        result.append({
            "key": key, "icon": icon, "title": title, "description": desc,
            "current": min(current, target), "target": target,
            "earned": current >= target,
        })
    return result


GARDEN_STAGES = [
    (0, "Empty plot", "?"),
    (1, "Seed", "?"),
    (5, "Sprout", "?"),
    (15, "Sapling", "?"),
    (30, "Young tree", "?"),
    (60, "Grove", "???"),
]


def garden_stage(total_events):
    """Garden stage from the number of recorded (rewarded) events."""
    total_events = max(int(total_events or 0), 0)
    index = 0
    for i, (threshold, _, _) in enumerate(GARDEN_STAGES):
        if total_events >= threshold:
            index = i
    threshold, name, emoji = GARDEN_STAGES[index]
    if index + 1 < len(GARDEN_STAGES):
        next_threshold, next_name, _ = GARDEN_STAGES[index + 1]
        span = next_threshold - threshold
        progress = (total_events - threshold) / span
        remaining = next_threshold - total_events
    else:
        next_name, remaining, progress = None, 0, 1.0
    return {
        "name": name, "emoji": emoji, "next_name": next_name,
        "remaining": remaining, "progress": max(0.0, min(progress, 1.0)),
    }


def daily_missions(today_events, daily_goal, bins_used_this_session, advice_this_session):
    """Three achievable missions. Scope is stated so nothing is overclaimed.

    Mission 1 uses the saved database (today). Missions 2 and 3 only count
    what happened in this browser session.
    """
    today_events = int(today_events or 0)
    return [
        {"title": f"Record {daily_goal} items today", "scope": "saved today",
         "current": min(today_events, daily_goal), "target": daily_goal,
         "done": today_events >= daily_goal},
        {"title": "Sort items into 2 different bins", "scope": "this session",
         "current": min(bins_used_this_session, 2), "target": 2,
         "done": bins_used_this_session >= 2},
        {"title": "Read ReWA Buddy advice for 1 item", "scope": "this session",
         "current": min(advice_this_session, 1), "target": 1,
         "done": advice_this_session >= 1},
    ]
```

```


def passport_stamps(points_rows, bin_names):
    """One stamp per bin; stamped once that bin has earned points."""
    stamped = {r.get("category") for r in (points_rows or []) if (r.get("points") or 0) > 0}
    return [{"bin": name, "stamped": name in stamped} for name in bin_names]
```

## rewabuddy_advisor.py

```
import os
import time
import requests

from google import genai
from recycling_logic import decide_bin


# ============================================================
# SMART RECYCLING ASSISTANT - CONFIGURATION
# ============================================================

# NOTE: Check the exact model name in Google AI Studio.
# A wrong name makes Gemini always fail and fall back to Groq.
GEMINI_MODELS = [
    "gemini-3.8-flash",
]

GROQ_MODEL = "openai/gpt-oss-120b"

OPENROUTER_MODEL = "openrouter/free"

REQUEST_TIMEOUT = 30

MIN_CONFIDENCE_FOR_AUTOMATIC_SORTING = 0.55

# A bin at or above this simulated fill level counts as "nearly full"
# (same threshold the dashboard uses for its red warning).
BIN_FULL_THRESHOLD = 90


# ============================================================
# MATERIAL CLASSIFICATION (decided by Python, not by the AI)
# Matches the problem statement: recyclable, reusable,
# biodegradable, hazardous
# ============================================================

HAZARDOUS_KEYWORDS = [
    "battery", "e-waste", "electronic", "chemical", "bulb",
    "medicine", "paint", "aerosol", "syringe", "tube light",
    "cfl", "pesticide", "thermometer", "toner",
]

BIODEGRADABLE_KEYWORDS = [
    "food", "leaf", "leaves", "fruit", "vegetable", "organic",
    "peel", "garden", "eggshell", "tea leaves", "coffee grounds",
    "flower", "grass",
]

# If the label mentions packaging, it is NOT organic waste
# (e.g. "food wrapper" is packaging, not food).
PACKAGING_WORDS = ["plastic", "wrapper", "packet", "packaging", "bottle", "can"]


def classify_material(waste_type):
    """Return 'hazardous', 'biodegradable', or 'recyclable and reusable'."""
    label = str(waste_type).lower()

    if any(word in label for word in HAZARDOUS_KEYWORDS):
        return "hazardous"

    if any(word in label for word in BIODEGRADABLE_KEYWORDS):
        if not any(word in label for word in PACKAGING_WORDS):
            return "biodegradable"

    return "recyclable and reusable"


# ============================================================
# BIN DECISION FOR HAZARDOUS / BIODEGRADABLE ITEMS
# (decide_bin from recycling_logic.py still handles the
```

```
#  plastic / paper / metal / glass items)
# ============================================================

SPECIAL_BINS = {
    "hazardous": "Hazardous",
    "biodegradable": "Biodegradable",
}


def decide_special_bin(material_class, confidence, bin_status):
    """Python-only bin decision for hazardous and biodegradable items."""
    bin_name = SPECIAL_BINS[material_class]

    # bin_status keys are lowercase ("hazardous"), but be tolerant
    status_lookup = {
        str(key).lower(): value
        for key, value in (bin_status or {}).items()
    }
    info = status_lookup.get(material_class)

    fill = None
    if isinstance(info, dict):
        fill = info.get("fill_percent")

    is_full = fill is not None and fill >= BIN_FULL_THRESHOLD

    if material_class == "hazardous":
        note = (
            "Hazardous items need special handling. Keep them "
            "separate from ordinary waste and recycling."
        )
    else:
        note = (
            "Keep organic waste separate from dry recyclables "
            "so it can be composted."
        )

    return {
        "bin": bin_name,
        "fill_percent": fill,
        "is_full": is_full,
        "needs_verification": confidence < MIN_CONFIDENCE_FOR_AUTOMATIC_SORTING,
        "note": note,
    }


# ============================================================
# REUSE KNOWLEDGE BASE (written by us, so the AI never has to
# invent reuse or safety advice on its own)
# ============================================================

# --- Recyclable materials: 3 safe, cheap reuse ideas per item ---
ITEM_REUSE = {
    "plastic bottle": [
        "Cut the bottle and use the bottom half as a pen stand or a small planter.",
        "Poke holes in the cap and use it as a watering can for plants.",
        "Use a clean bottle to store dry items such as rice, pulses or loose stationery.",
    ],
    "plastic bag": [
        "Reuse it as a bin liner for small dustbins.",
        "Carry it again for groceries or keep it in your bag for wet clothes.",
        "Use it to protect books or shoes inside your backpack on rainy days.",
    ],
    "plastic wrapper": [
        "Cut clean wrappers into strips for craft projects or collages.",
        "Use a clean wrapper to wrap small items when packing a parcel.",
        "Collect clean wrappers for school or college art projects.",
    ],
    "plastic container": [
        "Wash it well and use it to store snacks, screws, buttons or stationery.",
        "Use it as a tray under a plant pot.",
        "Use it as a drawer organiser for small items.",
    ],
    "cardboard box": [
        "Flatten it and reuse it for packing, or keep it for moving house.",
```

```
        "Cut it into drawer dividers or a desk organiser.",
        "Use it as a base for crafts, models or a college project.",
    ],
    "paper sheet": [
        "Use the blank side for rough work or notes.",
        "Cut clean sheets into small notepads or sticky-note style pads.",
        "Use it for origami, paper crafts or gift wrapping.",
    ],
    "newspaper": [
        "Use it to wrap fragile items or gifts.",
        "Use it to line drawers or shelves.",
        "Crumple it to clean glass and mirrors, or use it to cover surfaces while painting.",
    ],
    "metal can": [
        "Wash it, cover any sharp edge with tape, and use it as a pen holder.",
        "Use it as a small planter after making drainage holes carefully.",
        "Decorate it and use it as a desk organiser for small tools.",
    ],
    "glass bottle": [
        "Wash it and use it as a water bottle for the fridge or a flower vase.",
        "Use it to store homemade items like pickles or juice, only if it is properly cleaned.",
        "Decorate it as a lamp base or a desk decoration.",
    ],
    "glass jar": [
        "Wash it and use it to store spices, pulses or snacks.",
        "Use it as a pen holder or a small planter for succulents.",
        "Use it as a candle holder or decoration.",
    ],
}

# --- Fallback by material keyword when the exact label is unknown ---
CATEGORY_REUSE = [
    (["plastic"], ITEM_REUSE["plastic container"]),
    (["cardboard", "carton", "box"], ITEM_REUSE["cardboard box"]),
    (["paper"], ITEM_REUSE["paper sheet"]),
    (["metal", "tin", "can", "aluminium", "aluminum"], ITEM_REUSE["metal can"]),
    (["glass", "jar"], ITEM_REUSE["glass jar"]),
]

GENERIC_REUSE = [
    "Check whether the item can be reused for storage or as a container.",
    "Donate it if it is still in good condition.",
    "Repurpose it for a craft or college project before discarding.",
]

# --- Biodegradable items: composting and soil-friendly ideas ---
BIODEGRADABLE_REUSE = {
    "food waste": [
        "Add vegetable and fruit scraps to a home compost pit or a compost bin.",
        "Try a small kitchen compost bucket if you live in a hostel or flat.",
        "Take it to a community compost point if your area has one. Do not mix it with plastic.",
    ],
    "fruit peel": [
        "Chop banana or fruit peels and bury them in plant soil to enrich it.",
        "Make an eco-enzyme cleaner by fermenting fruit peels with jaggery and water.",
        "Add the peels to a compost bin or bucket.",
    ],
    "leaves": [
        "Use dry leaves as mulch around plants to keep the soil moist.",
        "Add dry leaves to a compost pile to balance wet kitchen waste.",
        "Use clean dry leaves for pressed-leaf art or crafts.",
    ],
}

BIODEGRADABLE_DEFAULT = [
    "Compost it at home or in a community compost bin.",
    "Bury small amounts in plant soil so they break down naturally.",
    "Keep it separate from plastic, metal and other dry waste.",
]

# --- Hazardous items: NO DIY reuse, only safe handling ---
HAZARDOUS_GUIDE = {
    "battery": {
        "reuse_line": (
            "Do not reuse or open used batteries. Switching to rechargeable "
```

```
            "batteries in future reduces battery waste."
        ),
        "handling": [
            "Never throw batteries in the normal dustbin or in a fire.",
            "Tape the ends of button or lithium cells and keep them dry.",
            "If a battery is swollen, leaking or hot, do not charge or puncture it. "
            "Place it in a non-metal container and take it to a collection point.",
        ],
    },
    "light bulb": {
        "reuse_line": (
            "Do not reuse a damaged or used bulb. A working bulb can be used "
            "in another fitting."
        ),
        "handling": [
            "CFL and tube lights contain mercury, so do not put them in normal waste.",
            "Wrap an intact bulb in paper or its box before taking it to a collection point.",
            "If one breaks, ventilate the room and pick up pieces with cardboard "
            "and gloves. Do not use a vacuum cleaner.",
        ],
    },
    "medicine": {
        "reuse_line": (
            "Do not reuse or share leftover or expired medicines. Return them "
            "through a take-back option."
        ),
        "handling": [
            "Return unused or expired medicines to a pharmacy or take-back point.",
            "Do not flush medicines unless the label specifically says so.",
            "Keep them in the original packaging, away from children and pets, "
            "until you dispose of them.",
        ],
    },
    "default": {
        "reuse_line": (
            "This item is hazardous, so do not try to reuse it yourself. "
            "Take it to a special collection point."
        ),
        "handling": [
            "Keep it separate from ordinary waste and recycling.",
            "Use an authorised hazardous-waste or e-waste collection point.",
            "Keep it in its original container and away from heat, children and pets.",
        ],
    },
}


def get_reuse_info(waste_type, material_class):
    """
    Return reuse ideas and safety tips picked by Python.

    {
        "ideas":    [...],  # reuse / composting ideas (empty for hazardous)
        "reuse_line": str or None,  # short line for hazardous items
        "handling": [...],  # safe-handling tips (hazardous only)
    }
    """
    label = str(waste_type).strip().lower()

    if material_class == "hazardous":
        guide = HAZARDOUS_GUIDE["default"]
        for key in ("battery", "light bulb", "medicine"):
            if key.split()[-1] in label:
                guide = HAZARDOUS_GUIDE[key]
                break
        if "tube" in label or "cfl" in label:
            guide = HAZARDOUS_GUIDE["light bulb"]
        return {
            "ideas": [],
            "reuse_line": guide["reuse_line"],
            "handling": guide["handling"],
        }

    if material_class == "biodegradable":
        ideas = BIODEGRADABLE_REUSE.get(label)
```

```
        if ideas is None:
            if "leaf" in label or "leaves" in label:
                ideas = BIODEGRADABLE_REUSE["leaves"]
            elif "peel" in label or "fruit" in label:
                ideas = BIODEGRADABLE_REUSE["fruit peel"]
            elif "food" in label or "vegetable" in label:
                ideas = BIODEGRADABLE_REUSE["food waste"]
            else:
                ideas = BIODEGRADABLE_DEFAULT
        return {"ideas": ideas, "reuse_line": None, "handling": []}

    # Recyclable and reusable
    if label in ITEM_REUSE:
        ideas = ITEM_REUSE[label]
    else:
        ideas = GENERIC_REUSE
        for words, candidate in CATEGORY_REUSE:
            if any(word in label for word in words):
                ideas = candidate
                break

    return {"ideas": ideas, "reuse_line": None, "handling": []}


# ============================================================
# PROVIDER 1 - GEMINI
# ============================================================

def try_gemini(prompt):
    api_key = os.getenv("GEMINI_API_KEY")

    if not api_key:
        raise RuntimeError("GEMINI_API_KEY is not set.")

    client = genai.Client(api_key=api_key)
    last_error = None

    for model in GEMINI_MODELS:
        for attempt in range(2):
            try:
                response = client.models.generate_content(
                    model=model,
                    contents=prompt,
                )

                if not response.text:
                    raise RuntimeError(
                        "Gemini returned an empty response."
                    )

                return response.text, f"Gemini ({model})"

            except Exception as error:
                last_error = error

                # Retry temporary server errors once.
                error_text = str(error)

                temporary_error = any(
                    code in error_text
                    for code in ["429", "500", "502", "503", "504"]
                )

                if temporary_error and attempt == 0:
                    time.sleep(2)
                    continue

                break

    raise RuntimeError(f"Gemini failed: {last_error}")


# ============================================================
# SHARED OPENAI-COMPATIBLE API CALL
# Used by Groq and OpenRouter
```

```
# ============================================================

def call_openai_style(url, key_env, model, prompt, label):
    api_key = os.getenv(key_env)

    if not api_key:
        raise RuntimeError(f"{key_env} is not set.")

    if not model:
        raise RuntimeError(f"{label} model is not configured.")

    headers = {
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json",
    }

    # OpenRouter supports these optional headers.
    if "openrouter.ai" in url:
        headers["HTTP-Referer"] = "http://localhost:8051"
        headers["X-Title"] = "EcoSort AI"

    payload = {
        "model": model,
        "messages": [
            {
                "role": "system",
                "content": (
                    "You are EcoSort AI, a careful recycling and "
                    "reuse assistant. "
                    "Use only the facts supplied in the prompt. "
                    "Never invent bin levels, measurements, or facts. "
                    "Explain uncertainty clearly."
                ),
            },
            {
                "role": "user",
                "content": prompt,
            },
        ],
        "temperature": 0.3,
    }

    response = requests.post(
        url,
        headers=headers,
        json=payload,
        timeout=REQUEST_TIMEOUT,
    )

    response.raise_for_status()

    data = response.json()

    try:
        answer = data["choices"][0]["message"]["content"]
    except (KeyError, IndexError, TypeError) as error:
        raise RuntimeError(
            f"{label} returned an unexpected response."
        ) from error

    if isinstance(answer, list):
        answer = "\n".join(
            part.get("text", "")
            for part in answer
            if isinstance(part, dict)
        )

    if not isinstance(answer, str) or not answer.strip():
        raise RuntimeError(f"{label} returned an empty response.")

    return answer.strip(), f"{label} ({model})"


# ============================================================
# PROVIDER 2 - GROQ
```

```
# ============================================================

def try_groq(prompt):
    return call_openai_style(
        url="https://api.groq.com/openai/v1/chat/completions",
        key_env="GROQ_API_KEY",
        model=GROQ_MODEL,
        prompt=prompt,
        label="Groq",
    )


# ============================================================
# PROVIDER 3 - OPENROUTER
# ============================================================

def try_openrouter(prompt):
    return call_openai_style(
        url="https://openrouter.ai/api/v1/chat/completions",
        key_env="OPENROUTER_API_KEY",
        model=OPENROUTER_MODEL,
        prompt=prompt,
        label="OpenRouter",
    )


# ============================================================
# AI PROVIDER FALLBACK
# Gemini ? Groq ? OpenRouter
# ============================================================

def ask_ai(prompt):
    errors = []

    providers = [
        try_gemini,
        try_groq,
        try_openrouter,
    ]

    for provider in providers:
        try:
            answer, source = provider(prompt)

            print(f"EcoSort AI response generated by {source}")

            return answer, source

        except Exception as error:
            message = str(error)

            # Avoid printing API keys or request headers.
            print(
                f"{provider.__name__} failed: "
                f"{message[:250]}"
            )

            errors.append(
                f"{provider.__name__}: {message[:250]}"
            )

    raise RuntimeError(
        "All configured AI providers failed. "
        + " | ".join(errors)
    )


# ============================================================
# PYTHON-ONLY FALLBACK
# Used when all AI providers fail
# ============================================================

def offline_advice(waste_type, decision, material_class="recyclable and reusable"):
    reuse_info = get_reuse_info(waste_type, material_class)

```

```
    lines = [
        "## ?? Recycling Advice",
        f"Detected item: **{waste_type}** (class: {material_class}).",
    ]

    if decision.get("needs_verification", True):
        lines.append(
            "Please verify the material before deciding "
            "which bin to use."
        )

    if decision.get("note"):
        lines.append(decision["note"])

    lines.append("## ?? Recommended Bin")

    bin_name = decision.get("bin")

    if bin_name is None:
        lines.append(
            "No matching bin is available in this system."
        )
    else:
        fill = decision.get("fill_percent")

        if fill is None:
            lines.append(
                f"The suggested bin is **{bin_name}**."
            )
        else:
            lines.append(
                f"The suggested bin is **{bin_name}** "
                f"(simulated fill: {fill}%)."
            )

        if decision.get("is_full"):
            lines.append(
                "This bin is nearly full. Consider another "
                "appropriate collection point or report it."
            )

    # Reuse section depends on the material class.
    lines.append("## ? Reuse Ideas")

    if material_class == "hazardous":
        lines.append(reuse_info["reuse_line"])
    elif material_class == "biodegradable":
        lines.append(
            "This item is biodegradable. Compost it instead of "
            "throwing it away.\n"
            + "\n".join(f"- {idea}" for idea in reuse_info["ideas"])
        )
    else:
        lines.append(
            "\n".join(f"- {idea}" for idea in reuse_info["ideas"])
        )

    lines.append("## ? Smart Suggestion")

    if material_class == "hazardous":
        tips = reuse_info["handling"]
    else:
        tips = [
            "Check the item's material marking.",
            "Keep recyclable materials clean when required.",
            "Follow your local recycling rules.",
        ]

    lines.append("\n".join(f"- {tip}" for tip in tips))

    return "\n\n".join(lines)


# ============================================================
# MAIN RECYCLING ADVISOR
```

```
# Compatible with the CURRENT app1.py call:
#
# generate_recycling_advice(
#     waste_type=best_label,
#     confidence=best_score,
#     bin_status=bin_status
# )
# ============================================================

def generate_recycling_advice(
    waste_type,
    confidence=None,
    bin_status=None,
    score=None,
    bins=None,
):
    """
    Generate recycling AND reuse advice using AI providers
    with a Python-only fallback.

    Supports both calling styles:

    1. Current app1.py:
       waste_type, confidence, bin_status

    2. Earlier advisor interface:
       waste_type, score, bins
    """

    # Support the older argument names as well.
    if score is not None:
        confidence = score

    if bins is not None:
        bin_status = bins

    if confidence is None:
        confidence = 0.0

    if bin_status is None:
        bin_status = {}

    # The CLIP score is a similarity score, not a
    # calibrated probability of correctness.
    try:
        confidence = float(confidence)
    except (TypeError, ValueError):
        confidence = 0.0

    confidence = max(0.0, min(confidence, 1.0))

    # Python decides the material class; the AI only explains it.
    material_class = classify_material(waste_type)

    # Python selects the bin.
    if material_class in SPECIAL_BINS:
        decision = decide_special_bin(
            material_class,
            confidence,
            bin_status,
        )
    else:
        decision = decide_bin(
            waste_type,
            confidence,
            bin_status,
        )

    # Python also selects the reuse ideas / safety tips.
    reuse_info = get_reuse_info(waste_type, material_class)

    if material_class == "hazardous":
        reuse_block = (
            "- Reuse line (hazardous, no DIY reuse): "
            f"{reuse_info['reuse_line']}\n"
```

```
            "- Safe handling tips:\n"
            + "\n".join(f"  * {tip}" for tip in reuse_info["handling"])
        )
    else:
        reuse_block = "\n".join(
            f"- {idea}" for idea in reuse_info["ideas"]
        )

    prompt = f"""
You are EcoSort AI, a friendly recycling and reuse assistant
for first-year college students.

Explain the following checked facts in simple English.

FACTS FROM PYTHON:
- Detected image label: {waste_type}
- Material class: {material_class}
- CLIP similarity score: {confidence:.3f}
- This score is NOT a calibrated probability.
- Suggested bin: {decision.get("bin")}
- Bin fill level: {decision.get("fill_percent")}
- Bin readings are simulated, not measured by real sensors.
- Bin is nearly full: {decision.get("is_full")}
- Material verification needed: {decision.get("needs_verification")}
- Additional note: {decision.get("note")}

REUSE / SAFETY INFORMATION FROM PYTHON:
{reuse_block}

RULES:
1. Do not claim the image classification is certainly correct.
2. If verification is needed, tell the user to check the material.
3. Use only the facts provided above.
4. Never invent a bin level or sensor reading.
5. If a value is unavailable, say that it is unavailable.
6. If the suggested bin is nearly full, recommend reporting it
   or finding another appropriate collection point.
7. Mention that recycling rules vary by location.
8. Give practical advice suitable for a college student.
9. Do not change the bin selected by Python.
10. Under Reuse Ideas, use the ideas supplied by Python above.
    You may reword them in simple English, but do not add
    unsafe, expensive or tool-heavy ideas.
11. Reuse ideas must be safe, cheap, and doable by a college
    student with no special tools.
12. If the material class is hazardous (battery, e-waste, bulbs,
    medicine, chemicals), do NOT suggest DIY reuse. Under Reuse
    Ideas write only the one short reuse line supplied by Python,
    and use the safe handling tips for Smart Suggestion.
13. If the material class is biodegradable (food, leaves, peels),
    present the composting ideas supplied by Python as the reuse
    ideas.

Use exactly these four headings:

## ?? Recycling Advice

## ?? Recommended Bin

## ? Reuse Ideas

## ? Smart Suggestion

Under Reuse Ideas, give exactly three short ideas
(except for hazardous items, see rule 12).
Under Smart Suggestion, provide exactly three short tips.
"""

    try:
        answer, source = ask_ai(prompt)

        return (
            f"{answer}\n\n"
            f"*Answered by: {source}*"
        )
```

```
747
748     except Exception as error:
749         print(
750             "All AI providers failed; using Python fallback:",
751             str(error)[:300],
752         )
753
754         return (
755             offline_advice(waste_type, decision, material_class)
756             + "\n\n"
757             "*AI providers are unavailable. "
758             "This answer was generated using Python rules.*"
759         )
760
761
762 # ============================================================
763 # OPTIONAL STANDALONE TEST
764 # ============================================================
765
766 if __name__ == "__main__":
767     test_bins = {
768         "plastic": {"fill_percent": 45, "source": "simulated"},
769         "paper": {"fill_percent": 70, "source": "simulated"},
770         "metal": {"fill_percent": 25, "source": "simulated"},
771         "glass": {"fill_percent": 92, "source": "simulated"},
772         "hazardous": {"fill_percent": 15, "source": "simulated"},
773         "biodegradable": {"fill_percent": 60, "source": "simulated"},
774     }
775
776     for item in ["plastic bottle", "battery", "fruit peel"]:
777         print("=" * 60)
778         print(item)
779         print(
780             generate_recycling_advice(
781                 waste_type=item,
782                 confidence=0.87,
783                 bin_status=test_bins,
784             )
785         )
```

## rewards.py

```
"""
rewards.py - Eco Points, daily activity and streaks for EcoSortAI.

Per-user version: every row now belongs to a user_id (set by
data_base.set_current_user() at the top of each script run).

Design notes
------------
* Uses the SAME SQLite file as data_base.py, but only adds its own two tables.
* Points are awarded with INSERT OR IGNORE on a UNIQUE event_key, so a rerun,
  refresh, or double click can never award the same event twice.
* The reward row and the daily-activity row are written in ONE transaction.
* Streaks are computed from the saved dates, never from session state.
* Column names deliberately avoid "item_name"/"material" so that the
  "Clear recent events" button does not delete earned points.
* Existing databases are migrated automatically; old rows are kept under
  the user id "legacy" until claimed with claim_legacy_rewards().
"""

import sqlite3
from datetime import date, timedelta

import data_base

POINTS_PER_EVENT = 10
DAILY_GOAL = 5  # recycled items per day that count as a full daily goal
LEGACY_USER = data_base.LEGACY_USER


def db_path():
    """Locate the same database file used by data_base.py."""
    return (
        getattr(data_base, "DB_NAME", None)
        or getattr(data_base, "DB_PATH", None)
        or getattr(data_base, "DATABASE", None)
        or "recycling.db"
    )


def _connect():
    conn = sqlite3.connect(db_path())
    conn.row_factory = sqlite3.Row
    return conn


def _uid():
    # Raises if nobody is signed in, so data can never leak between users.
    return data_base._uid()


def _columns(conn, table):
    return [row[1] for row in conn.execute(f'PRAGMA table_info("{table}")')]


def init_rewards_db():
    """Create / migrate the reward tables (safe to call every run)."""
    conn = _connect()
    try:
        with conn:
            conn.execute(
                """
                CREATE TABLE IF NOT EXISTS reward_events (
                    id            INTEGER PRIMARY KEY AUTOINCREMENT,
                    event_key     TEXT NOT NULL UNIQUE,
                    item_label    TEXT NOT NULL,
                    bin_name      TEXT NOT NULL,
                    points        INTEGER NOT NULL,
                    activity_date TEXT NOT NULL,
                    created_at    TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
                )
                """
```

```
            )
            if "user_id" not in _columns(conn, "reward_events"):
                conn.execute(
                    "ALTER TABLE reward_events "
                    f"ADD COLUMN user_id TEXT NOT NULL DEFAULT '{LEGACY_USER}'"
                )
            conn.execute(
                "CREATE INDEX IF NOT EXISTS idx_reward_user "
                "ON reward_events(user_id)"
            )

            # daily_activity used to be keyed by date alone. SQLite cannot
            # change a primary key, so an old table is rebuilt once.
            existing = _columns(conn, "daily_activity")
            if existing and "user_id" not in existing:
                conn.execute("ALTER TABLE daily_activity RENAME TO daily_activity_old")
                existing = []
            if not existing:
                conn.execute(
                    """
                    CREATE TABLE daily_activity (
                        user_id       TEXT NOT NULL,
                        activity_date TEXT NOT NULL,
                        events        INTEGER NOT NULL DEFAULT 0,
                        points        INTEGER NOT NULL DEFAULT 0,
                        PRIMARY KEY (user_id, activity_date)
                    )
                    """
                )
            has_old = conn.execute(
                "SELECT 1 FROM sqlite_master WHERE name = 'daily_activity_old'"
            ).fetchone()
            if has_old:
                conn.execute(
                    f"""
                    INSERT INTO daily_activity (user_id, activity_date, events, points)
                    SELECT '{LEGACY_USER}', activity_date, events, points
                    FROM daily_activity_old
                    """
                )
                conn.execute("DROP TABLE daily_activity_old")
    finally:
        conn.close()


def award_event(event_key, item_label, bin_name, today=None):
    """
    Award POINTS_PER_EVENT for one successfully recorded recycling event.

    Returns True if points were awarded, False if this event_key was already
    rewarded (duplicate). Raises on real database errors.
    """
    uid = _uid()
    today = (today or date.today()).isoformat()
    conn = _connect()
    try:
        with conn:  # one atomic transaction
            cur = conn.execute(
                """
                INSERT OR IGNORE INTO reward_events
                    (event_key, item_label, bin_name, points, activity_date, user_id)
                VALUES (?, ?, ?, ?, ?, ?)
                """,
                (event_key, item_label, bin_name, POINTS_PER_EVENT, today, uid),
            )
            if cur.rowcount == 0:
                return False  # already rewarded

            conn.execute(
                """
                INSERT INTO daily_activity (user_id, activity_date, events, points)
                VALUES (?, ?, 1, ?)
                ON CONFLICT(user_id, activity_date) DO UPDATE SET
                    events = events + 1,
                    points = points + excluded.points
```

```
                """,
                (uid, today, POINTS_PER_EVENT),
            )
        return True
    finally:
        conn.close()


def _calculate_streaks(day_strings, today):
    """Return (current_streak, longest_streak) from a list of ISO date strings."""
    days = sorted({date.fromisoformat(d) for d in day_strings})
    if not days:
        return 0, 0

    longest = run = 0
    previous = None
    for d in days:
        run = run + 1 if previous and (d - previous).days == 1 else 1
        longest = max(longest, run)
        previous = d

    current = run if (today - days[-1]).days <= 1 else 0
    return current, longest


def get_summary(today=None):
    """Everything the rewards dashboard needs, in one dict."""
    uid = _uid()
    today = today or date.today()
    conn = _connect()
    try:
        totals = conn.execute(
            "SELECT COALESCE(SUM(points), 0) AS pts, COUNT(*) AS evts "
            "FROM reward_events WHERE user_id = ?",
            (uid,),
        ).fetchone()
        day_rows = conn.execute(
            "SELECT activity_date FROM daily_activity "
            "WHERE user_id = ? ORDER BY activity_date",
            (uid,),
        ).fetchall()
        today_row = conn.execute(
            "SELECT events, points FROM daily_activity "
            "WHERE user_id = ? AND activity_date = ?",
            (uid, today.isoformat()),
        ).fetchone()
    finally:
        conn.close()

    current, longest = _calculate_streaks([r["activity_date"] for r in day_rows], today)
    return {
        "total_points": totals["pts"],
        "total_events": totals["evts"],
        "active_days": len(day_rows),
        "today_events": today_row["events"] if today_row else 0,
        "today_points": today_row["points"] if today_row else 0,
        "current_streak": current,
        "longest_streak": longest,
    }


def get_points_by_category():
    """List of dicts: category, events, points (highest points first)."""
    conn = _connect()
    try:
        rows = conn.execute(
            """
            SELECT bin_name AS category, COUNT(*) AS events, SUM(points) AS points
            FROM reward_events
            WHERE user_id = ?
            GROUP BY bin_name
            ORDER BY points DESC
            """,
            (_uid(),),
        ).fetchall()
```

```
    finally:
        conn.close()
    return [dict(r) for r in rows]


def get_daily_history(days=7, today=None):
    """Last `days` days (oldest first), with zeros for days without activity."""
    uid = _uid()
    today = today or date.today()
    start = today - timedelta(days=days - 1)
    conn = _connect()
    try:
        rows = conn.execute(
            """
            SELECT activity_date, events, points FROM daily_activity
            WHERE user_id = ? AND activity_date >= ?
            """,
            (uid, start.isoformat()),
        ).fetchall()
    finally:
        conn.close()

    lookup = {r["activity_date"]: r for r in rows}
    history = []
    for offset in range(days):
        d = start + timedelta(days=offset)
        row = lookup.get(d.isoformat())
        history.append(
            {
                "date": d.strftime("%a %d %b"),
                "events": row["events"] if row else 0,
                "points": row["points"] if row else 0,
            }
        )
    return history


def reset_rewards():
    """Delete ONLY the signed-in user's points, streaks and rewarded events.

    app.py's clear_all_rewards() looks for a function with this name first,
    so its "empty every table" fallback is never used.
    """
    uid = _uid()
    conn = _connect()
    try:
        with conn:
            conn.execute("DELETE FROM reward_events WHERE user_id = ?", (uid,))
            conn.execute("DELETE FROM daily_activity WHERE user_id = ?", (uid,))
    finally:
        conn.close()


# ---------- one-time adoption of pre-login rewards ----------
def count_legacy_rewards():
    conn = _connect()
    try:
        return conn.execute(
            "SELECT COUNT(*) FROM reward_events WHERE user_id = ?", (LEGACY_USER,)
        ).fetchone()[0]
    finally:
        conn.close()


def claim_legacy_rewards(user_id):
    """Move all pre-login points and daily activity to one account."""
    conn = _connect()
    try:
        with conn:
            conn.execute(
                "UPDATE reward_events SET user_id = ? WHERE user_id = ?",
                (user_id, LEGACY_USER),
            )
            old = conn.execute(
                "SELECT activity_date, events, points FROM daily_activity "
```

```
                "WHERE user_id = ?",
                (LEGACY_USER,),
            ).fetchall()
            for row in old:
                conn.execute(
                    """
                    INSERT INTO daily_activity (user_id, activity_date, events, points)
                    VALUES (?, ?, ?, ?)
                    ON CONFLICT(user_id, activity_date) DO UPDATE SET
                        events = events + excluded.events,
                        points = points + excluded.points
                    """,
                    (user_id, row["activity_date"], row["events"], row["points"]),
                )
            conn.execute("DELETE FROM daily_activity WHERE user_id = ?", (LEGACY_USER,))
    finally:
        conn.close()
```

## test_rewa.py

```
"""Tests for rewa_logic.py.

Run with either:
    python -m unittest -v test_rewa_logic
    python -m pytest -q          (if you have pytest installed)
"""
import io
import unittest

from PIL import Image

import rewa_logic as L


# ---------- helpers ----------
class FakeRewards:
    """Mimics the real rewards DB: one reward per event key."""

    def __init__(self, fail=False):
        self.keys = set()
        self.points = 0
        self.fail = fail

    def award(self, uid, label, category):
        if self.fail:
            raise RuntimeError("database is locked")
        if uid in self.keys:
            return False
        self.keys.add(uid)
        self.points += 10
        return True


def make_env(accept=True, rewards=None):
    rewards = rewards or FakeRewards()
    bins = []

    def record_fn(category, label):
        if not accept:
            return False
        bins.append((category, label))
        return True

    env = {
        "recorded_uids": set(),
        "recorded_bins": {},
        "pending_rewards": {},
        "record_fn": record_fn,
        "award_fn": rewards.award,
    }
    return env, rewards, bins


def run(env, uid="a", label="plastic bottle", category="Plastic"):
    return L.record_and_reward(uid, label, category, **env)


def png_bytes(size=(20, 20), fmt="PNG"):
    buf = io.BytesIO()
    Image.new("RGB", size, (10, 120, 60)).save(buf, format=fmt)
    return buf.getvalue()


class RecordingAndRewards(unittest.TestCase):
    def test_record_awards_points_once(self):
        env, rewards, bins = make_env()
        self.assertEqual(run(env), ("ok", ""))
        self.assertEqual(rewards.points, 10)
        self.assertEqual(bins, [("Plastic", "plastic bottle")])
        self.assertEqual(env["recorded_bins"]["a"], "Plastic")

```

```
    def test_double_click_does_not_double_record_or_reward(self):
        env, rewards, bins = make_env()
        run(env)
        self.assertEqual(run(env)[0], "duplicate")
        self.assertEqual(rewards.points, 10)
        self.assertEqual(len(bins), 1)

    def test_full_bin_gives_no_points_and_is_not_marked_recorded(self):
        env, rewards, _ = make_env(accept=False)
        self.assertEqual(run(env)[0], "full")
        self.assertEqual(rewards.points, 0)
        self.assertNotIn("a", env["recorded_uids"])

    def test_record_crash_gives_no_points(self):
        env, rewards, _ = make_env()

        def boom(category, label):
            raise OSError("disk error")

        env["record_fn"] = boom
        self.assertEqual(run(env)[0], "record_err")
        self.assertEqual(rewards.points, 0)
        self.assertNotIn("a", env["recorded_uids"])

    def test_points_failure_is_remembered_and_retry_recovers(self):
        rewards = FakeRewards(fail=True)
        env, _, _ = make_env(rewards=rewards)
        self.assertEqual(run(env)[0], "points_err")
        self.assertIn("a", env["pending_rewards"])
        self.assertEqual(rewards.points, 0)

        rewards.fail = False
        recovered = L.retry_pending_rewards(env["pending_rewards"], env["award_fn"])
        self.assertEqual(recovered, 1)
        self.assertEqual(rewards.points, 10)
        self.assertEqual(env["pending_rewards"], {})

    def test_retry_never_double_rewards(self):
        env, rewards, _ = make_env()
        run(env)
        env["pending_rewards"]["a"] = ("plastic bottle", "Plastic")  # stale entry
        L.retry_pending_rewards(env["pending_rewards"], env["award_fn"])
        self.assertEqual(rewards.points, 10)

    def test_category_correction_uses_confirmed_bin(self):
        env, _, bins = make_env()
        run(env, label="glass jar", category="Plastic")  # user corrected the bin
        self.assertEqual(bins, [("Plastic", "glass jar")])
        self.assertEqual(env["recorded_bins"]["a"], "Plastic")

    def test_same_photo_cannot_be_recorded_twice(self):
        env, rewards, bins = make_env()
        env["recorded_fingerprints"] = set()
        env["fingerprint"] = "photo-1"
        self.assertEqual(run(env, uid="a")[0], "ok")
        # same picture, classified again -> brand new uid, same fingerprint
        self.assertEqual(run(env, uid="b")[0], "same_photo")
        self.assertEqual(rewards.points, 10)
        self.assertEqual(len(bins), 1)

    def test_different_photos_are_both_recorded(self):
        env, rewards, _ = make_env()
        env["recorded_fingerprints"] = set()
        env["fingerprint"] = "photo-1"
        run(env, uid="a")
        env["fingerprint"] = "photo-2"
        self.assertEqual(run(env, uid="b")[0], "ok")
        self.assertEqual(rewards.points, 20)

    def test_failed_record_does_not_burn_the_photo(self):
        env, rewards, _ = make_env(accept=False)
        env["recorded_fingerprints"] = set()
        env["fingerprint"] = "photo-1"
        self.assertEqual(run(env, uid="a")[0], "full")
        self.assertEqual(env["recorded_fingerprints"], set())
```

```
147
148     def test_reward_gap(self):
149         self.assertEqual(L.reward_gap(5, 3), 2)
150         self.assertEqual(L.reward_gap(3, 5), 0)
151
152
153 class Uncertainty(unittest.TestCase):
154     def test_low_score_needs_manual_category(self):
155         self.assertTrue(L.needs_manual_category(0.30, "Plastic"))
156         self.assertTrue(L.needs_manual_category(0.90, None))
157         self.assertFalse(L.needs_manual_category(0.90, "Plastic"))
158
159
160 class ImageValidation(unittest.TestCase):
161     def test_valid_png_loads_as_rgb(self):
162         self.assertEqual(L.load_validated_image(png_bytes()).mode, "RGB")
163
164     def test_empty_file_rejected(self):
165         with self.assertRaises(ValueError):
166             L.load_validated_image(b"")
167
168     def test_text_file_rejected(self):
169         with self.assertRaises(ValueError):
170             L.load_validated_image(b"hello, I am not an image")
171
172     def test_oversized_file_rejected(self):
173         with self.assertRaises(ValueError):
174             L.load_validated_image(png_bytes(), max_bytes=10)
175
176     def test_unsupported_format_rejected(self):
177         with self.assertRaises(ValueError):
178             L.load_validated_image(png_bytes(fmt="BMP"))
179
180     def test_large_image_is_shrunk(self):
181         img = L.load_validated_image(png_bytes(size=(3000, 1000)), max_side=1600)
182         self.assertLessEqual(max(img.size), 1600)
183
184
185 class RewardExtras(unittest.TestCase):
186     def test_no_badges_when_nothing_recorded(self):
187         badges = L.badge_progress({"total_events": 0, "longest_streak": 0}, [])
188         self.assertFalse(any(b["earned"] for b in badges))
189
190     def test_badges_follow_summary(self):
191         rows = [{"category": "Plastic", "points": 10},
192                 {"category": "Paper", "points": 10},
193                 {"category": "Hazardous", "points": 10}]
194         badges = {b["key"]: b for b in L.badge_progress(
195             {"total_events": 5, "longest_streak": 3}, rows)}
196         self.assertTrue(badges["first"]["earned"])
197         self.assertTrue(badges["five"]["earned"])
198         self.assertTrue(badges["streak3"]["earned"])
199         self.assertFalse(badges["streak7"]["earned"])
200         self.assertTrue(badges["explorer"]["earned"])
201         self.assertTrue(badges["hazard"]["earned"])
202         self.assertFalse(badges["twentyfive"]["earned"])
203
204     def test_garden_stages(self):
205         self.assertEqual(L.garden_stage(0)["name"], "Empty plot")
206         self.assertEqual(L.garden_stage(1)["name"], "Seed")
207         self.assertEqual(L.garden_stage(5)["name"], "Sprout")
208         self.assertEqual(L.garden_stage(14)["name"], "Sprout")
209         self.assertEqual(L.garden_stage(15)["name"], "Sapling")
210         top = L.garden_stage(500)
211         self.assertEqual(top["name"], "Grove")
212         self.assertIsNone(top["next_name"])
213
214     def test_garden_progress_between_stages(self):
215         stage = L.garden_stage(3)  # Seed(1) -> Sprout(5)
216         self.assertEqual(stage["remaining"], 2)
217         self.assertAlmostEqual(stage["progress"], 0.5)
218
219     def test_missions(self):
220         m = L.daily_missions(2, 3, 1, 0)
221         self.assertEqual([x["done"] for x in m], [False, False, False])
```

```
        m = L.daily_missions(5, 3, 2, 1)
        self.assertTrue(all(x["done"] for x in m))
        self.assertEqual(m[0]["current"], 3)  # capped at the target

    def test_passport_stamps(self):
        stamps = L.passport_stamps([{"category": "Glass", "points": 10}],
                                   ["Plastic", "Glass"])
        self.assertEqual(stamps, [{"bin": "Plastic", "stamped": False},
                                  {"bin": "Glass", "stamped": True}])


if __name__ == "__main__":
    unittest.main()
```